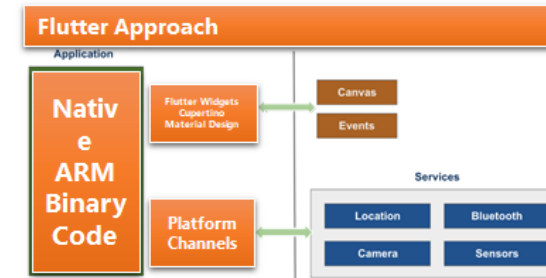
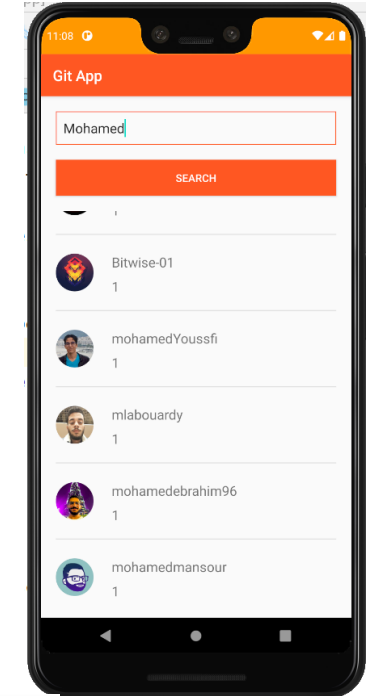
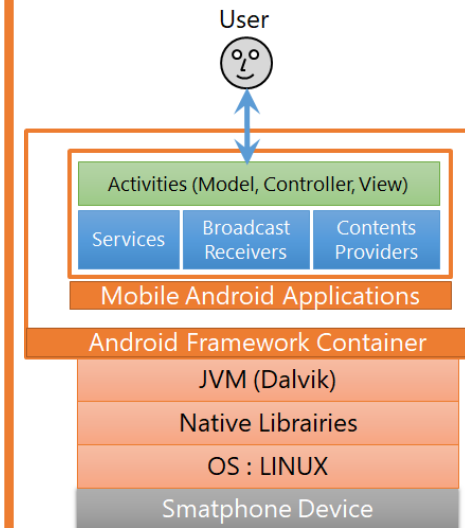
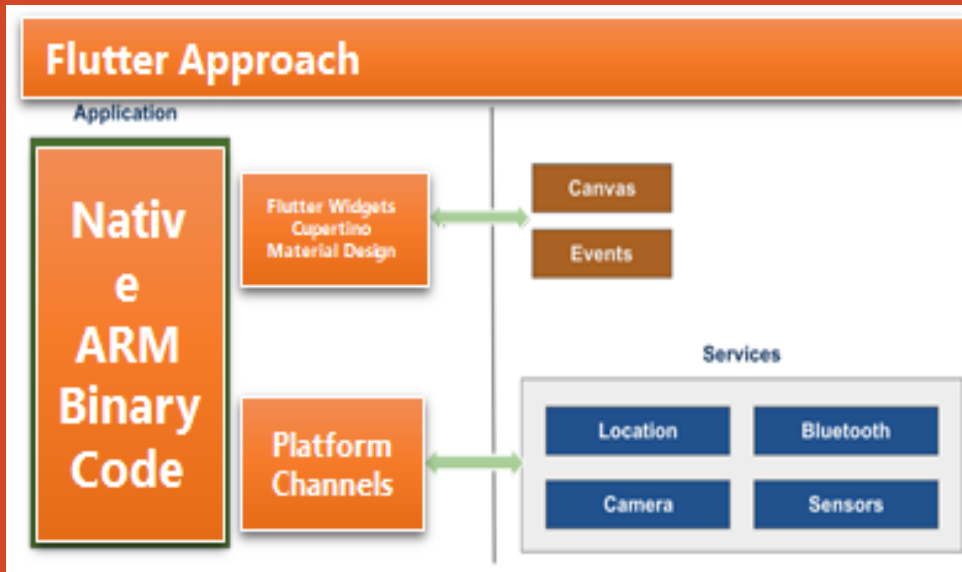


Développement Mobile

- Natif Android
- Cross Platform : Flutter



Jamal MAWANE

Laboratoire Informatique, Intelligence Artificielle et Cyber Sécurité (2IACS)

ENSET, Université Hassan II Casablanca, Maroc

Email : mawane@enset-media.ac.ma

Concepts de base



ANDROID

Concepts de base

Une application Android est composée des éléments suivants:

- Activité
- Services
- Récepteurs de diffusion (Broadcast Receivers)
- Fournisseurs de contenu (Content Providers)



Concepts de base

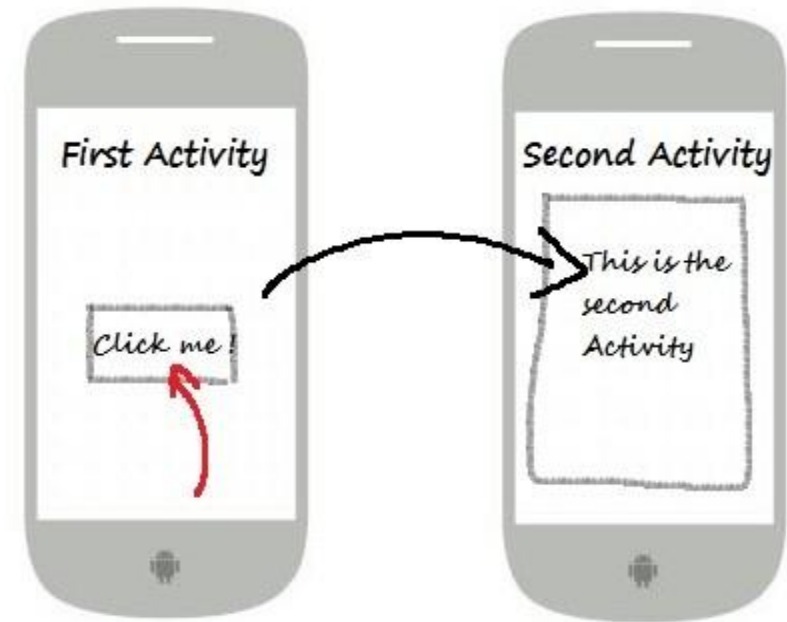
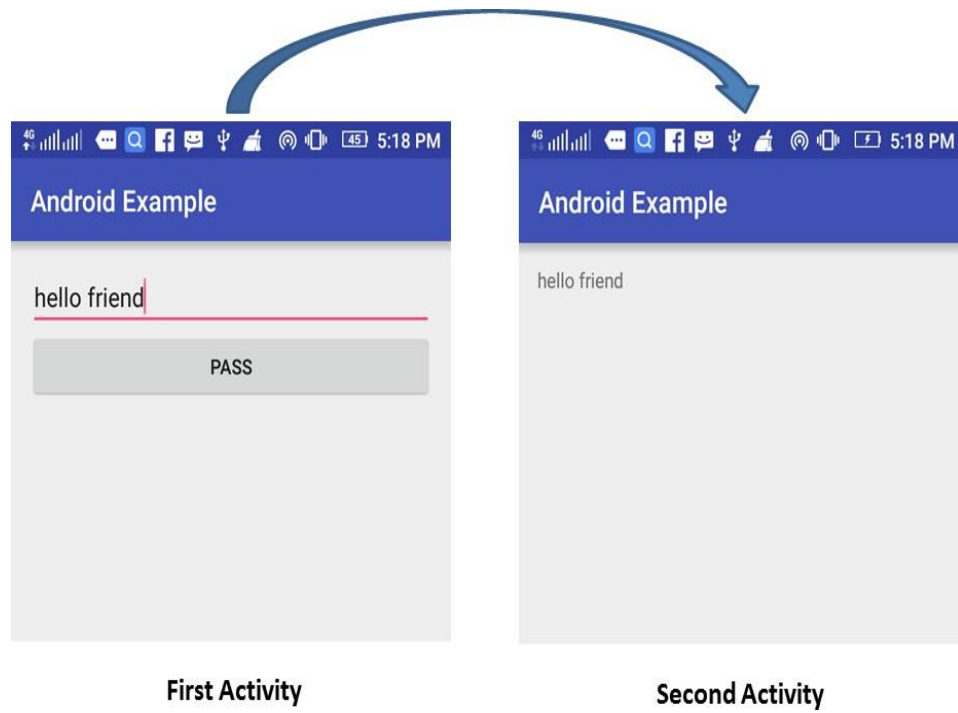
Activité: Définition

- **Élément de base** d'une interface utilisateur.
- Equivalent d'une fenêtre (Windows, Linux) ou d'une boîte de dialogue d'une application classique.
- Une application comportant plusieurs écrans, possédera donc autant d'activités.
- Une activité peut interagir: avec l'utilisateur en lui demandant des données, avec d'autres activités ou services en émettant des **intentions** ou fournissant du contenu.



Concepts de base

Activité: **Exemple**

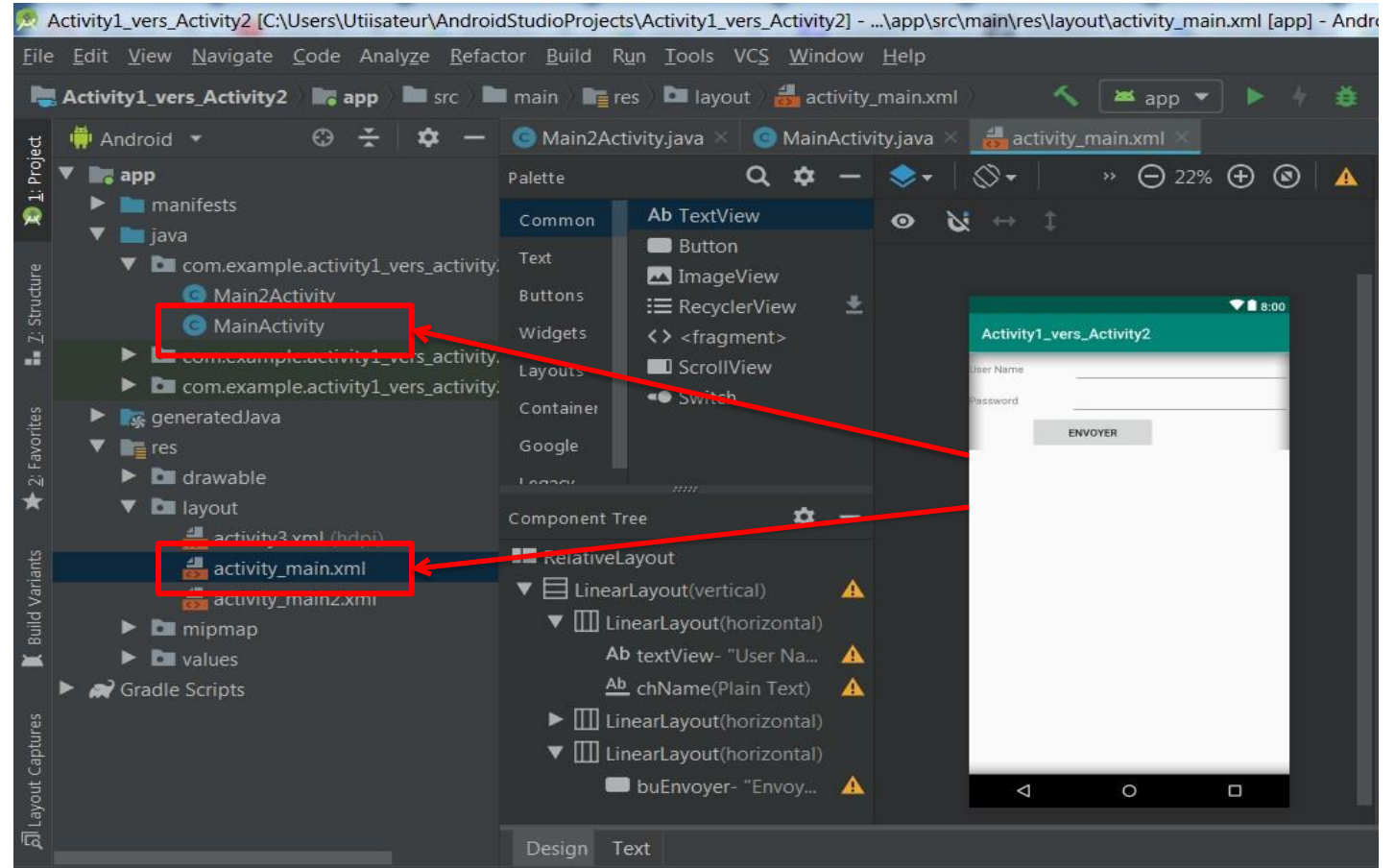


Concepts de base

Activité: Fichiers de base

Chaque activité est liée à trois fichiers:

1. Le fichier manifest où elle déclarée
2. Un fichier XML dans le répertoire /res/layouts qui décrit son design
3. Un fichier kotlin pour la contrôler



Concepts de base

Activité: Fichiers de base

- Pour déclarer une **Activité** dans le fichier **manifest.xml**, on utilise la balise **<activity>**.
- La balise **<intent-filter>** indique la première activité qui se lance au démarrage de l'application.

```
<activity android:name=".Main2Activity"></activity>
<activity android:name=".MainActivity">
  <intent-filter>
    <action android:name="android.intent.action.MAIN" />

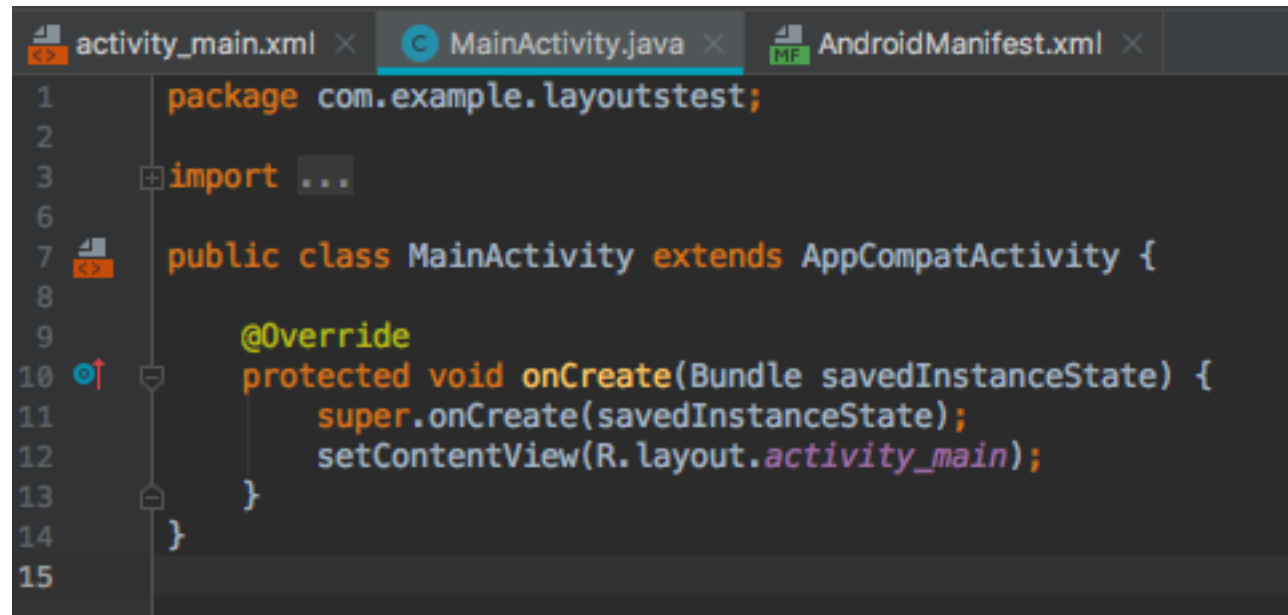
    <category android:name="android.intent.category.LAUNCHER" />
  </intent-filter>
</activity>
```



Concepts de base

Activité: Fichiers de base

- Les activités des applications Android sont implémentées en Kotlin par des *classe d'activités* : ce sont des sous-classes de **android.app.Activity**.



```
1 package com.example.layouttest;
2
3 import ...
4
5
6
7 public class MainActivity extends AppCompatActivity {
8
9     @Override
10    protected void onCreate(Bundle savedInstanceState) {
11        super.onCreate(savedInstanceState);
12        setContentView(R.layout.activity_main);
13    }
14 }
15
```



Concepts de base

Les services

- Un service est une opération exécutée en tâche de fond et fonctionne indépendamment de toute activité.
- Il permet à une application d'indiquer au SE qu'il a besoin d'effectuer une opération longue en dehors du cycle normal de l'Activité.

Exemple: le lecteur de la musique

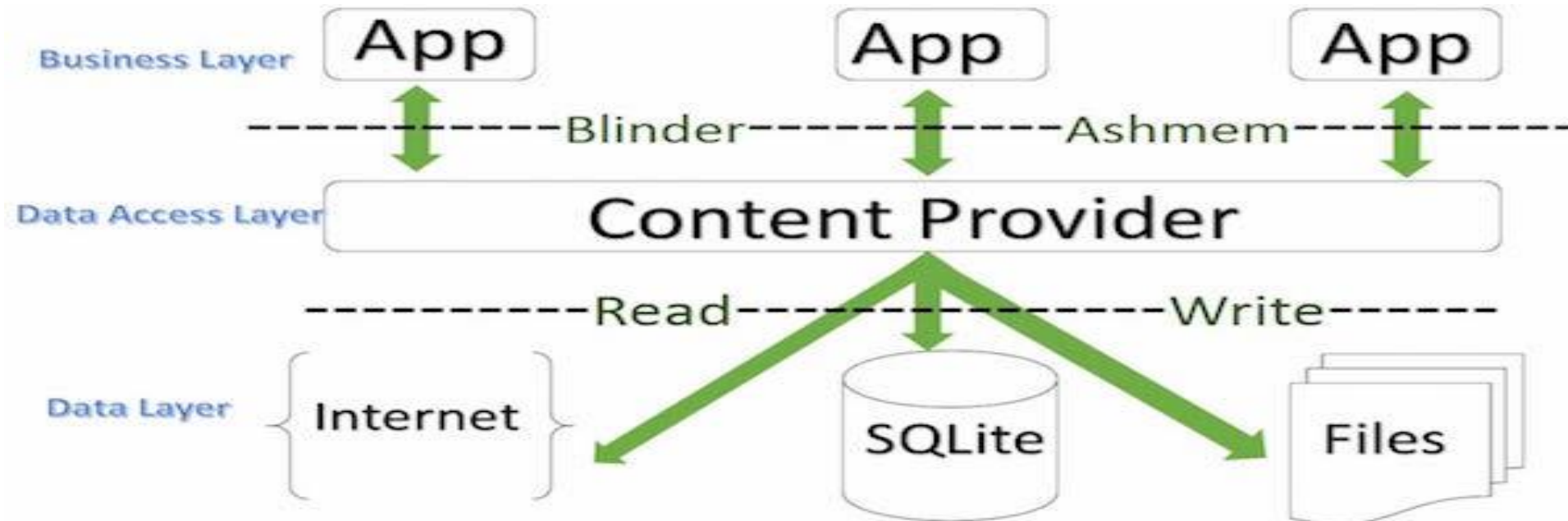


Concepts de base

Les fournisseurs de contenu

Le développement Android encourage le partage de données entre applications.

- Les **fournisseurs de contenu** permettent l'accès aux données: Contacts, Agenda, Photos , etc....



Concepts de base

Les récepteurs de diffusion

- Gèrent la communication entre le système d'exploitation Android et les applications qui se fait via des **intents**.
- Un **intent** est une "intention" à faire quelque chose contenant des informations destinées à un composant Android.

Le but des Intents est de:

1. Naviguer entre les activités.
2. Passer des paramètres d'une activité à une autre.
3. Ecouter et réagir à des événements extérieurs à l'application.



Interaction entre Kotlin et XML



ANDROID

Binding

Interaction entre Kotlin et XML

Kotlin vs. XML

Si le **visuel** est défini dans un fichier **XML**, toute la partie **interaction**, quant à elle, est définie dans des classes **kotlin**. Nous allons donc avoir deux problèmes à aborder :

1. Comment faire le lien entre kotlin et XML ?
2. Comment réagir à un événement ?



Interaction entre Kotlin et XML

Comment faire le lien entre Kotlin et XML ?

- Pour effectuer des modifications graphiques sur un composant, il faut utiliser un **identifiant XML** pour **récupérer une référence** vers ce composant.
- En kotlin, il faut utiliser la méthode **findViewById** pour récupérer la vue associée à un identifiant XML.



Le type View est le type parent de tous les composants graphiques.



Interaction entre Java et XML

Comment faire le lien entre Java et XML ?

Exemple:

- La simple méthode **findViewById** permettra de **récupérer une référence** vers un bouton nommé **mybtn** dans le fichier XML.
- Pour modifier le texte présent sur le bouton, on utilise la méthode **setText**.

```
1 Button btn = (Button)findViewById(R.id.mybtn);  
2 btn.setText("Texte modifié depuis Java");
```



Gestion des Événements



ANDROID

Gestion des Événements

Gestion des événements

➤ Il existe plusieurs façons d'interagir avec une interface graphique. Par exemple cliquer sur **un bouton ou une touche du clavier**, etc.

=> Ces interactions s'appellent des **événements**.

➤ **Méthode 1**: créer un auditeur d'événements et l'enregistrer auprès du composant (View).

➤ **Méthode 2** : les View sont elles mêmes auditrices de certains événements : (cliquer sur un bouton par exemple). Il suffit donc de spécialiser la méthode adaptée et lancée lorsque l'événement survient .



Gestion des Événements

Méthode 1: Création d'un auditeur d'événements

- Pour pouvoir réagir à l'apparition d'un événement, il faut utiliser un **objet qui va détecter l'évènement** afin de vous permettre **le traiter**.
=> Ce type d'objet s'appelle un **listener**.
- Un **listener** est une interface qui vous oblige à redéfinir des méthodes qui vont être appelée au moment où se produira l'évènement associé.



Gestion des Événements

Méthode 1: Création d'un auditeur d'événements

On va faire implémenter un **listener** à notre classe:

- Ce qui veut dire que notre classe sera une classe à **l'écoute des actions** qu'on effectuera sur les vues.
- A chaque fois qu'on effectuera une action, une méthode qui correspond à cette action sera appelée, et on définira dans cette méthode ce qu'il faut faire une fois l'action effectuée.



Gestion des Événements

Méthode 1: Création d'un auditeur d'événements

- Il existe plusieurs interfaces, une pour chaque type d'actions, nous allons voir **uniquement le clic normal** pour l'instant. Uniquement des méthodes relatives aux cliques pourront être implémentées.
- L'interface utilisée s'appelle **View.OnClickListener** et nous **oblige** à définir la méthode **void onClick (View v)**.
 - => C'est dans cette méthode qu'on définira la conduite à adopter en cas de clic. L'argument de type **View est la vue sur laquelle le clic a été effectué.**



Gestion des Événements

Méthode 1: Création d'un auditeur d'événements

Avec cette méthode, l'événement est réalisé dans la méthode **onCreate()** de l'activité suivant les étapes ci-dessous:

1. On récupère le composant depuis le layout XML ***findViewById(R.layout.id_composant)***.
2. On appelle le setter pour l'installation du listener.
3. On insère le corps d'événement à l'intérieur du listener.



Gestion des Événements

Méthode 1: Création d'un auditeur d'événements

Avec cette méthode, l'événement est réalisé dans la méthode ***onCreate()*** de l'activité suivant les étapes ci-dessous:

1. On récupère le composant depuis le layout XML ***findViewById(R.layout.id_composant)***.
2. On appelle le setter pour l'installation du listener.
3. On insère le corps d'événement à l'intérieur du listener.



Gestion des Événements

Méthode 1: Création d'un auditeur d'événements

```
public class MainActivity extends AppCompatActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
  
        final EditText txt1;  
        final EditText txt2;  
        final EditText txt3;  
        Button bu;  
        Button buReset;
```

```
        txt1 = (EditText) findViewById(R.id.editText1);  
        txt2 = (EditText) findViewById(R.id.editText2);  
        txt3 = (EditText) findViewById(R.id.editText3);  
        bu = (Button) findViewById(R.id.buttonSummit);  
        buReset = (Button) findViewById(R.id.buttonReset);  
  
        buReset.setOnClickListener(new View.OnClickListener() {  
            @Override  
            public void onClick(View v) {  
                txt1.setText("");  
                txt2.setText("");  
                txt3.setText("");  
            }  
        });  
    }  
}
```

Test_LinearLayout

Nombre 1:

Nombre 2:

Resultat :

SOMME RESET

Gestion des Événements

Méthode 2: **View auditrice d'événement**

- Cette méthode permet **d'écrire directement la gestion d'événements** qui peuvent se produire dans la View.
- L'attribut **android:onClick** d'un composant graphique indique le nom de la méthode qui sera lancée si on clique sur cette View.

Exemple:

```
<Button
    android:id="@+id/buttonReset"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginRight="20dp"
    android:layout_weight="1"
    android:onClick="buResetClick"
    android:text="Reset"/>
```

```
public void buResetClick(View view) {
    EditText txt1 = (EditText) findViewById(R.id.editText1);
    EditText txt2 = (EditText) findViewById(R.id.edittext2);
    EditText txt3 = (EditText) findViewById(R.id.editText3);
    txt1.setText("");
    txt2.setText("");
    txt3.setText("");
}
```


Gestion des Événements

Différents types d'événements

Principaux événements supportés (fonctionnement dépendant du type de composant et du dispositif d'entrée) :

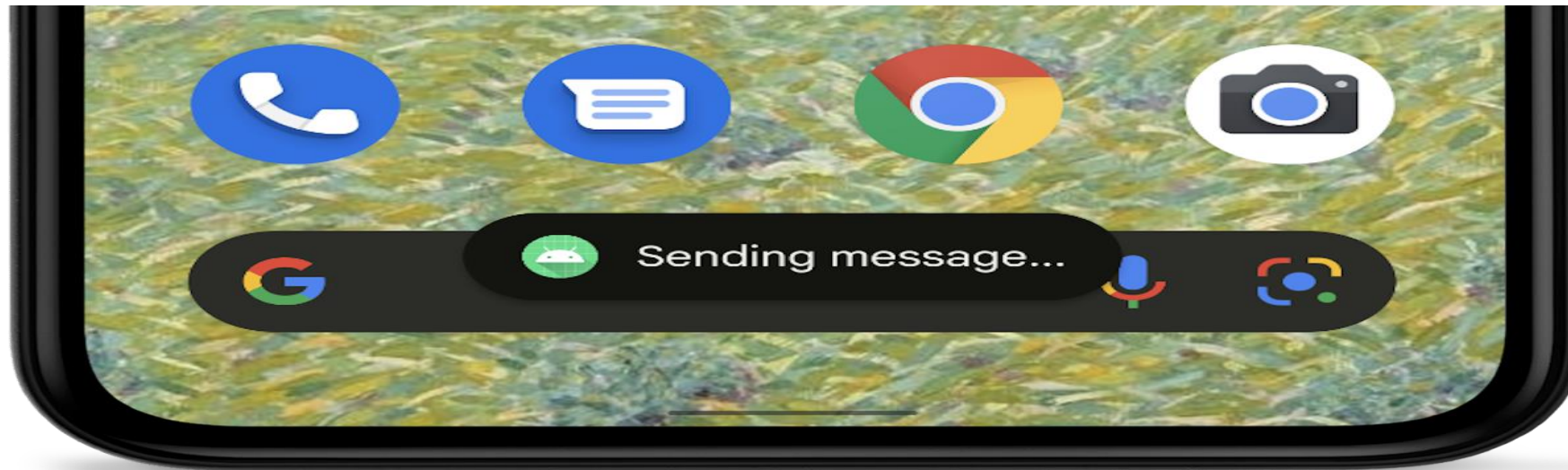
- **click** : clic sur un composant (appui puis relâchement rapide).
- **longClick** : clic long sur un composant.
- **key** : appui sur une touche d'un clavier physique
- **createContextMenu** : invoqué lors de la création d'un menu contextuel
- **drag** : événement de glissé (lors d'un glissé-déposé).
- ...



Gestion des Événements

Événement Toast

- La classe **Toast** permet de créer un texte qui apparaît en premier plan puis disparaît au bout d'un temps donné. A considérer comme un message d'information ou d'avertissement.
- Par exemple, si vous cliquez sur Envoyer dans un e-mail, un toast s'affiche et indique "Sending message" (Envoi du message), comme dans la capture d'écran ci-dessous :



Gestion des Événements

Événement Toast

➤ La classe Toast permet de créer un message avec la méthode **makeText()**, cette dernière prend 3 paramètres :

- **Le contexte de l'application** (this depuis une Activity),
- **Le message à afficher,**
- **La durée d'affichage** (La durée pourra prendre deux valeurs: `Toast.LENGTH_SHORT` ou `Toast.LENGTH_LONG`) .

Si votre application cible Android 12 (niveau d'API 31) ou une version ultérieure, son toast est limité à deux lignes de texte et affiche l'icône de l'application à côté du texte. Notez que la longueur d'une ligne varie en fonction de la taille de l'écran. Vous devez donc raccourcir le texte le plus possible.



Gestion des Événements

Événement Toast

Exemple:

```
Toast.makeText(Context, "Sending message",Toast.LENGTH_SHORT).show();
```

➤ La méthode *makeText* construit une instance de Toast avec ces paramètres que vous n'aurez plus qu'à afficher via la méthode *show()*.



Gestion des Événements

Événement Toast

Remarque:

Il est possible d'utiliser *makeText* avec un identifiant sur le texte à afficher en passant ainsi par une ressource texte:

`Toast.makeText (Context context, int resId, duration)`

Avec:

`resId = R.string.Nom_resource_string`



Activité n° 1

Enoncé : L'objectif de cette activité est de créer plusieurs interfaces graphiques et de gérer les évènements suivants :

Click

LongClick

Swipe (gauche et droite)

TextChange

OnItemSelected

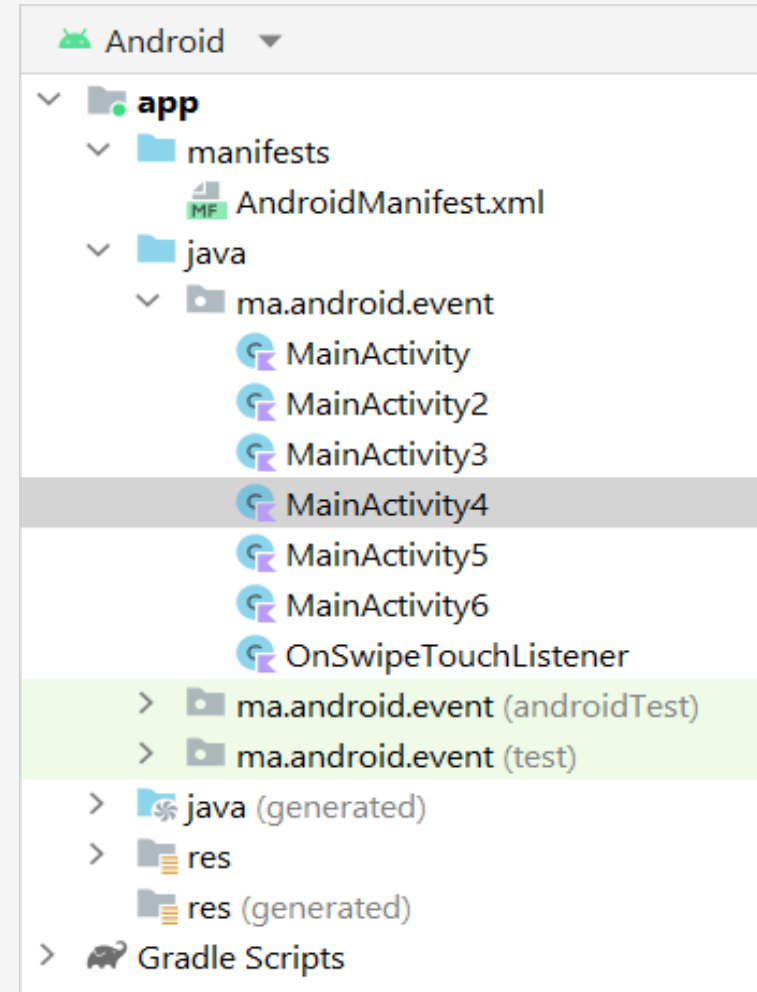
OnTouch

CreateContextMenu

OndateSet

La navigation entre les activités sera effectuée en utilisant les évènements Swipe (gauche et droite).

Structure de projet et Démonstration



Activité n° 1

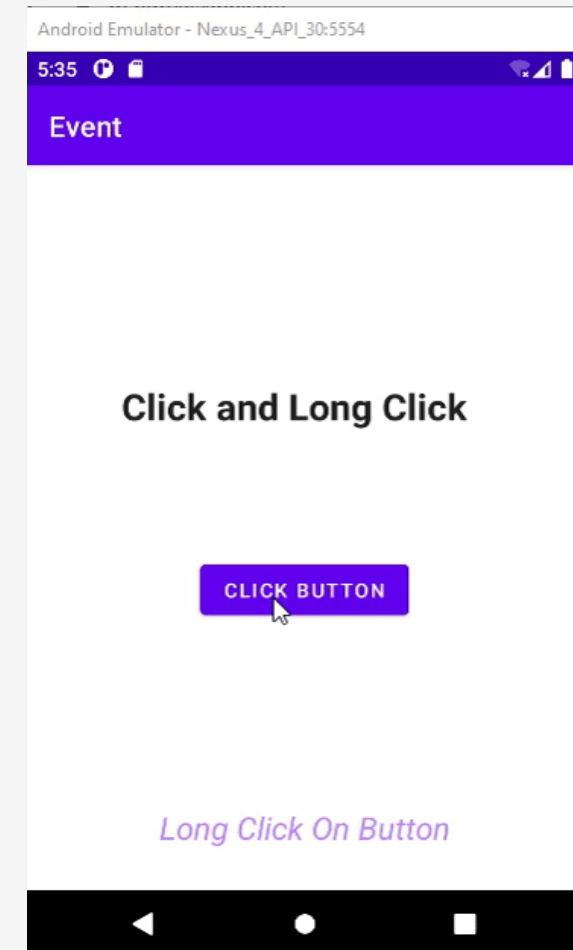
Création de plusieurs interfaces graphiques et gestion des évènements

Tâche 1 : Click et LongClick

Il est demandé de créer une activité (**MainActivity**) comprenant un bouton sur lequel seront appliqués les évènements de click suivants :

OnClickListener et
OnLongClickListener, .

comme illustré dans la figure ci-jointe



Activité n° 1

Création de plusieurs interfaces graphiques et gestion des évènements

Etape 1 : Modifier le fichier layout

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity"
    android:id="@+id/activity1">
    <Button
        android:id="@+id/click"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="276dp"
        android:text="Click Button"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintHorizontal_bias="0.498"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="136dp"
        android:text="Click and Long Click "
        android:textSize="26dp"
        android:textStyle="bold"
```

```
        style="@style/MaterialAlertDialog.MaterialComponents.Title.Icon"
        app:layout_constraintBottom_toTopOf="@+id/click"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintVertical_bias="0.16" />
    <TextView
        android:id="@+id/longText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="120dp"
        android:text=""
        android:textColor="@color/purple_200"
        android:textSize="22dp"
        android:textStyle="italic"
        android:padding="5dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/click"
        app:layout_constraintVertical_bias="0.141" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

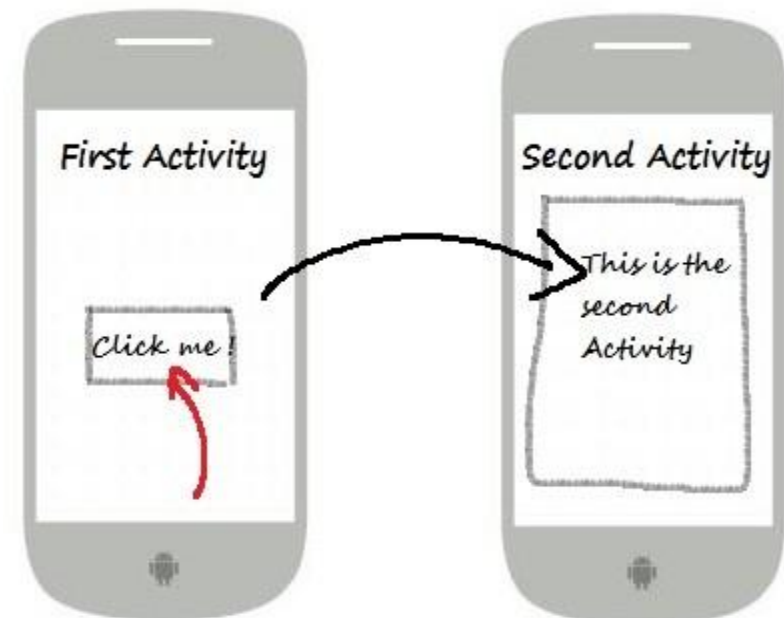
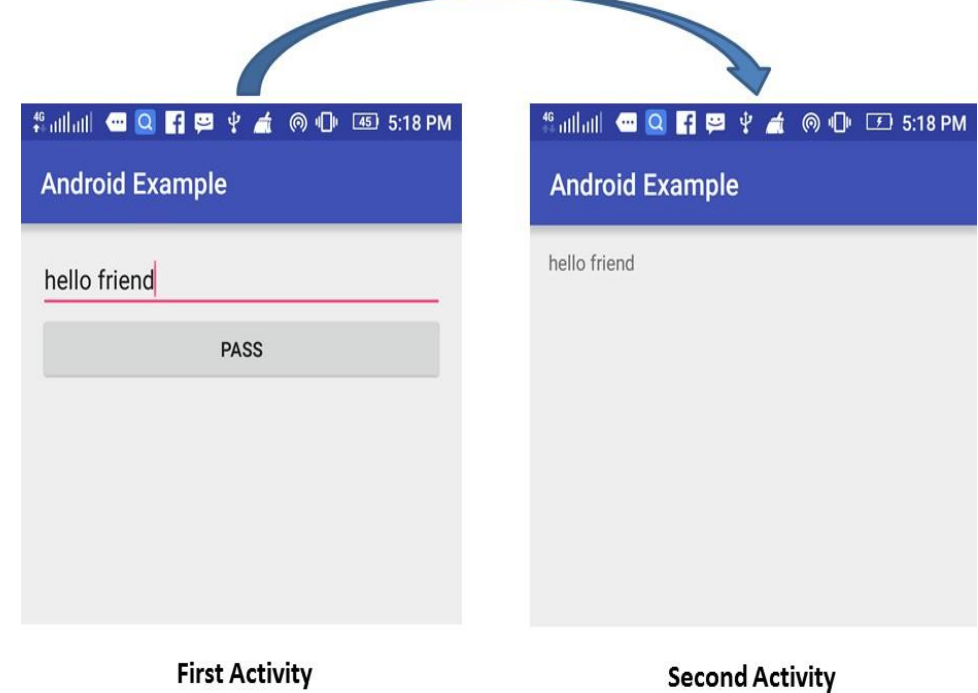

Activité n° 1

Création de plusieurs interfaces graphiques et gestion des évènements

Etape 2 : Modifier le code de la classe activité

```
class MainActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        val activity1 = findViewById<ConstraintLayout>(R.id.activity1)  
        var click = findViewById<Button>(R.id.click)  
        val longText = findViewById<TextView>(R.id.longText)  
  
        click.setOnClickListener {  
            Toast.makeText(this, "Button click ", Toast.LENGTH_SHORT).show()  
            longText.text = "Simple Click On Button "  
        }  
  
        click.setOnLongClickListener {  
            Toast.makeText(this@MainActivity, "Button Long click", Toast.LENGTH_SHORT).show()  
            longText.text = "Long Click On Button"  
            true  
        }  
    }  
}
```

La navigation entre les activités



Communication entre les activités

- La communication du système interne Android est basée sur l'envoi et la réception de messages exprimant l'intention d'effectuer une action.
- Chacun de ses messages peut-être émis à destination d'un autre composant de l'application (ex: un service) ou celui d'une autre application.
- Issu de la classe Intent, ce message permet donc de véhiculer toutes les informations nécessaires à la réalisation de l'action.

Communication entre les activités

Utilisation des intents

- Un **intent** est une "intention" à faire quelque chose contenant des informations destinées à un composant Android.
- Le but des Intents est de:
 1. Naviguer entre les activités.
 2. Passer des paramètres d'une activité à une autre.
 3. Ecouter et réagir à des événements extérieurs à l'application.

Communication entre les activités

Utilisation des intents

Pour passer d'une activité courante à une nouvelle activité, il faut:

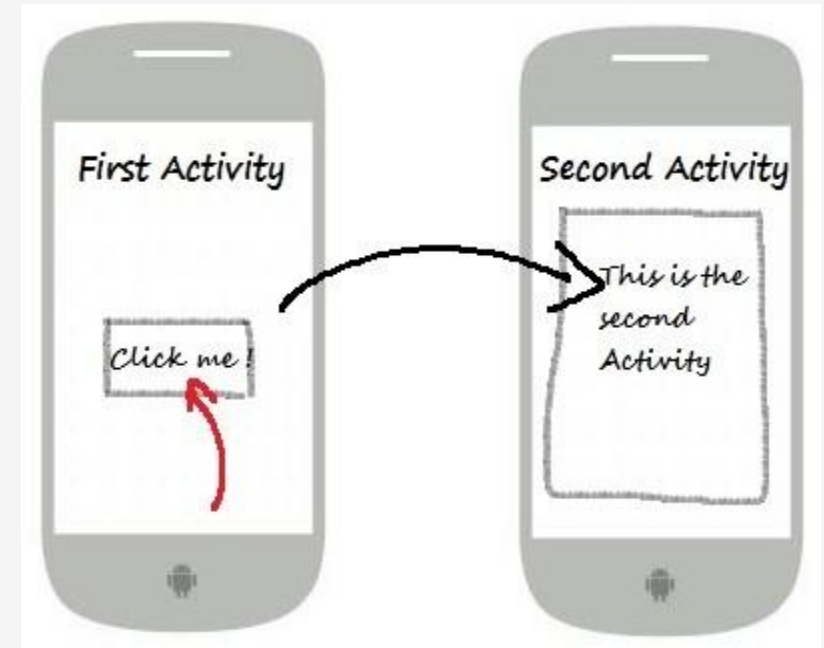
1. Ecrire le code :

```
Intent i = new Intent(getApplicationContext(), NouvelleActivity.class);
```

```
startActivity(i);
```

2. Déclarer la nouvelle activité `NouvelleActivity.class` (le futur écran) dans `AndroidManifest.xml`

startActivity: est une méthode de la classe `Activity` permettant de lancer une autre `Activity` (qui affichera un nouvel écran).



Communication entre les activités

Utilisation des intents

- La **transmission de données** s'effectue avec la classe **Bundle**.
- Cette classe est une sorte de **conteneur pour les données transmissibles d'une activité à l'autre**. Elle permet de récupérer tout type de données : *long, char, ArrayList* ...

Communication entre les activités

- La **transmission de données** s'effectue avec la classe **Bundle**.
- Cette classe est une sorte de **conteneur pour les données transmissibles d'une activité à l'autre**. Elle permet de récupérer tout type de données : *long, char, ArrayList ...*

1. Etape 1: le chargement des données dans un bundle dans la classe mère.

```
Intent myIntent = new Intent(this, Activity2.class);  
Bundle b = new Bundle();  
b.putString("Name", chName.getText().toString());  
myIntent.putExtras(b);
```

2. Etape 2: la récupération des données transmises à la classe cible.

```
Bundle b = getIntent().getExtras();  
String name = b.getString("Name");
```


La persistance des données



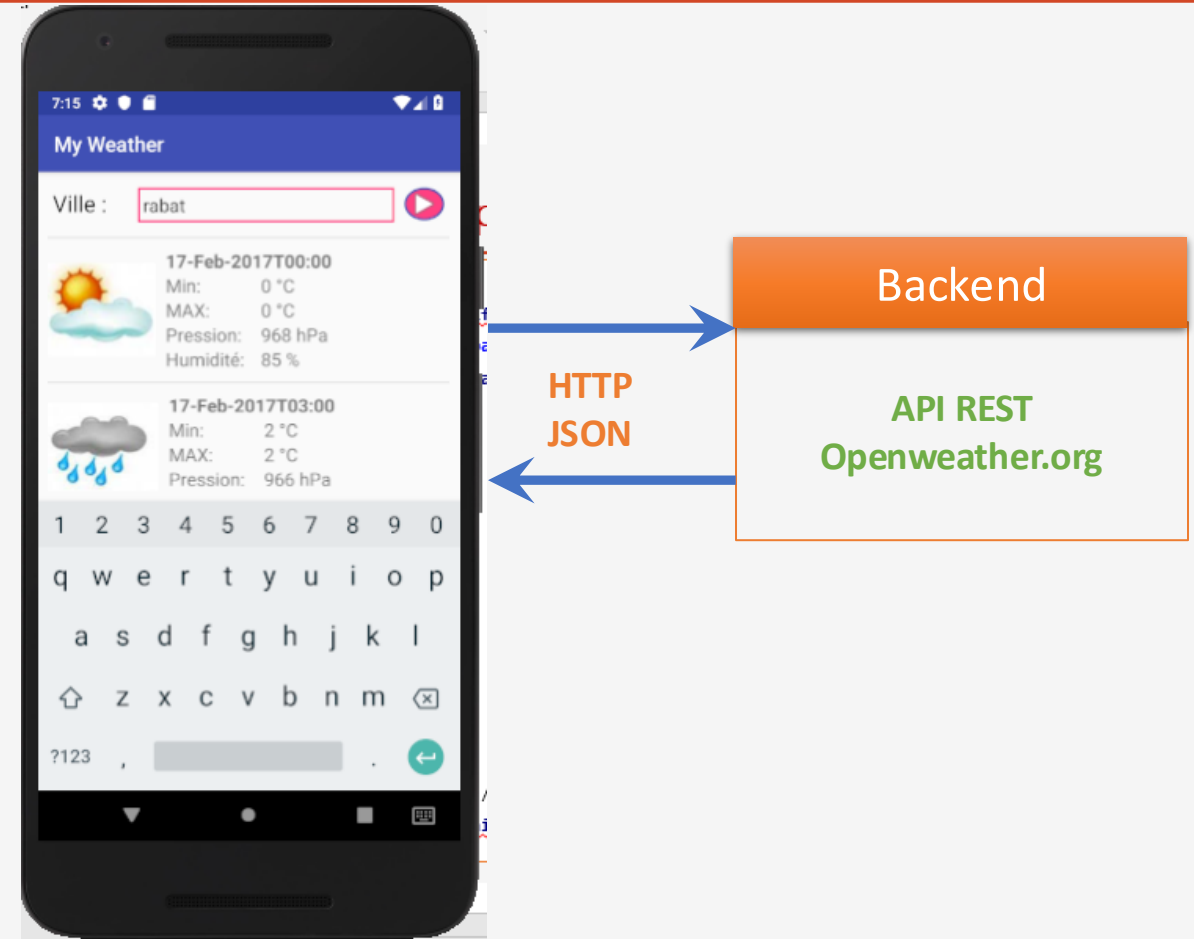
Communication entre les activités

Il existe essentiellement quatre façons différentes pour stocker les données dans une application Android:

- **Préférences Partagées (Shared Preferences)**
- **Système de fichiers**
- **Bases de données SQLITE**
- **Serveur distant**

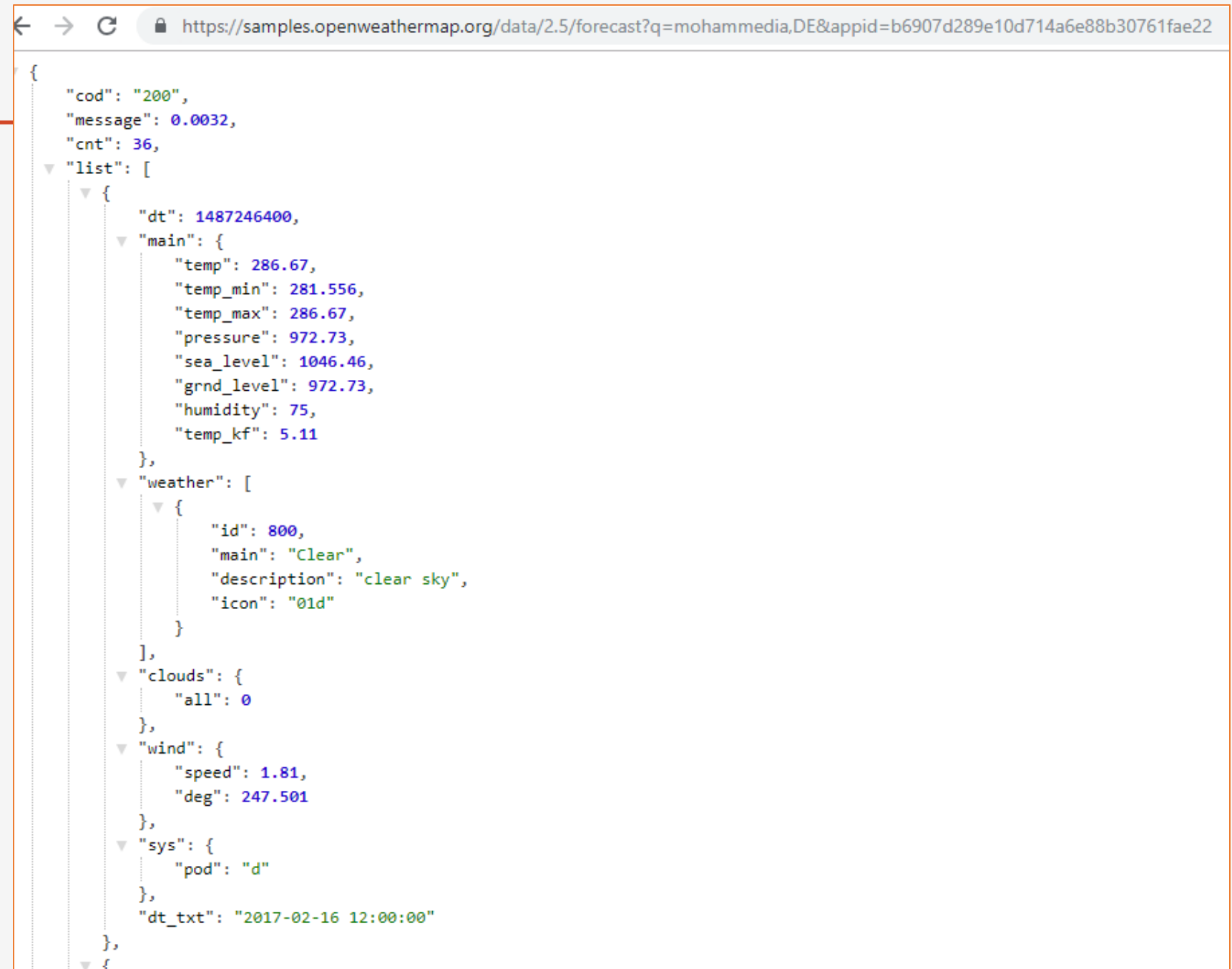
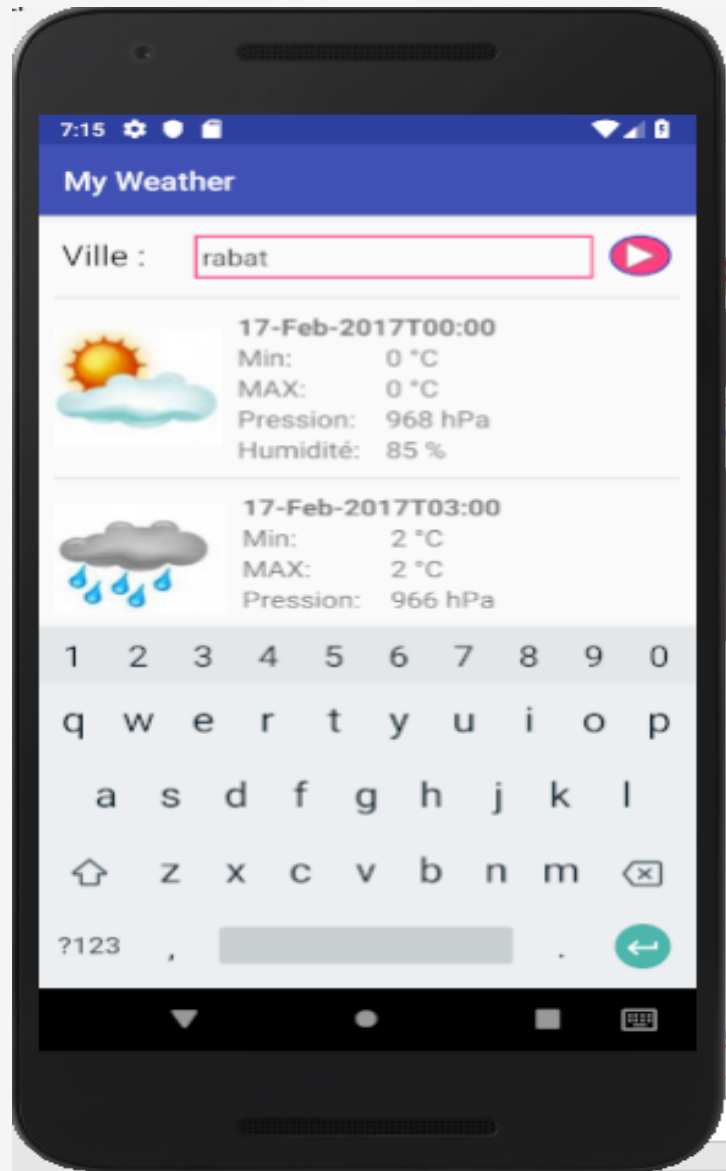
Activité n° 2 : Météo

- Créer une application mobile Android qui permet de :
 - Saisir une ville
 - Afficher les prévisions météo de cette ville en faisant appel à l'API REST exposé par openweather.org. On affichera pour chaque prévision :
 - Date et heure
 - Température maximale
 - Température minimale
 - Pression atmosphérique
 - Humidité



<https://samples.openweathermap.org/data/2.5/forecast?q=mohammedia,DE&appid=b6907d289e10d714a6e88b30761fae22>

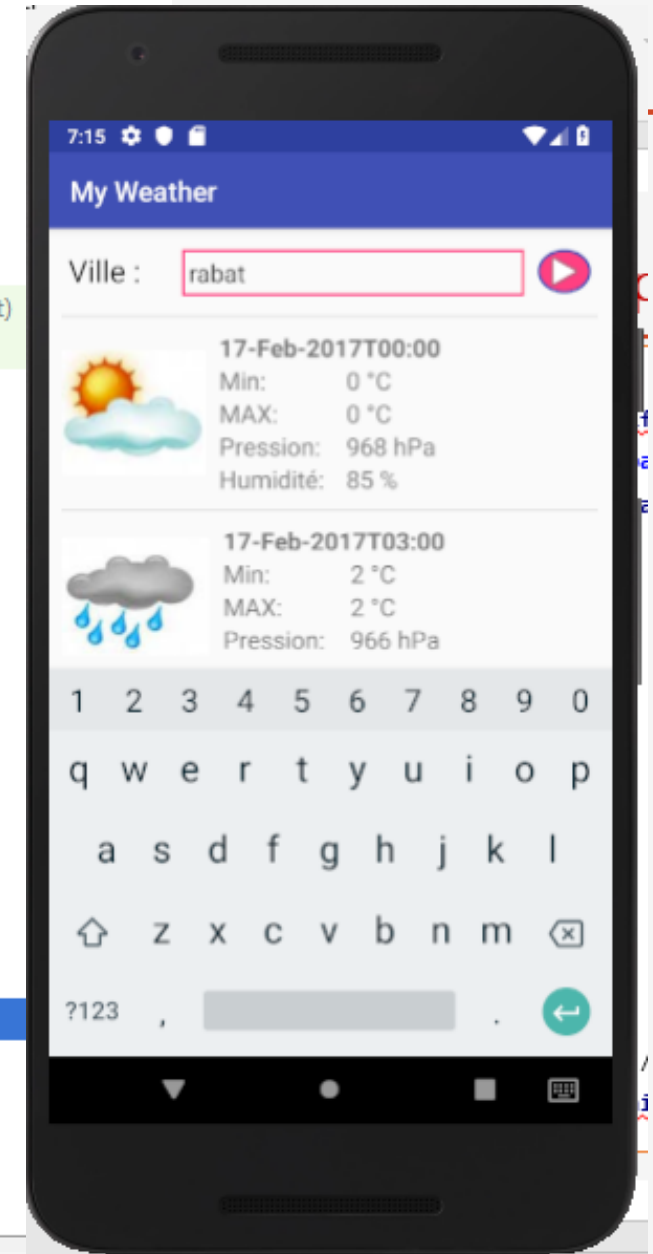
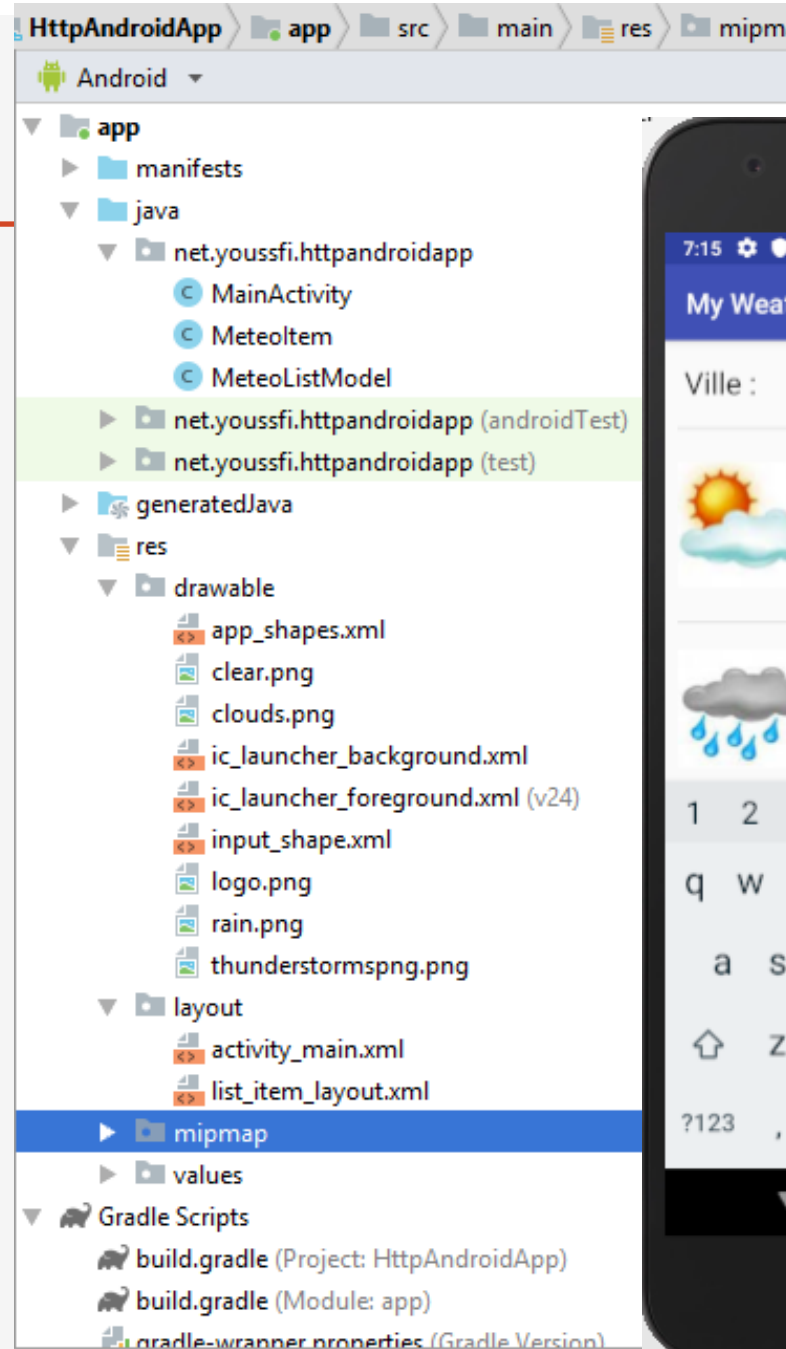
Openweather.org



Activité n° 2 : Météo

build.gradle

```
dependencies {  
    implementation  
        'com.android.volley:volley:1.1.0'  
        implementation fileTree(dir: 'libs', include: ['*.jar'])  
        implementation 'com.android.support:appcompat-v7:27.1.1'  
    implementation  
        'com.android.support.constraint:constraint-layout:1.1.3'  
    testImplementation 'junit:junit:4.12'  
    androidTestImplementation  
        'com.android.support.test:runner:1.0.2'  
        'com.android.support.test.espresso:espresso-core:3.0.2'  
}
```

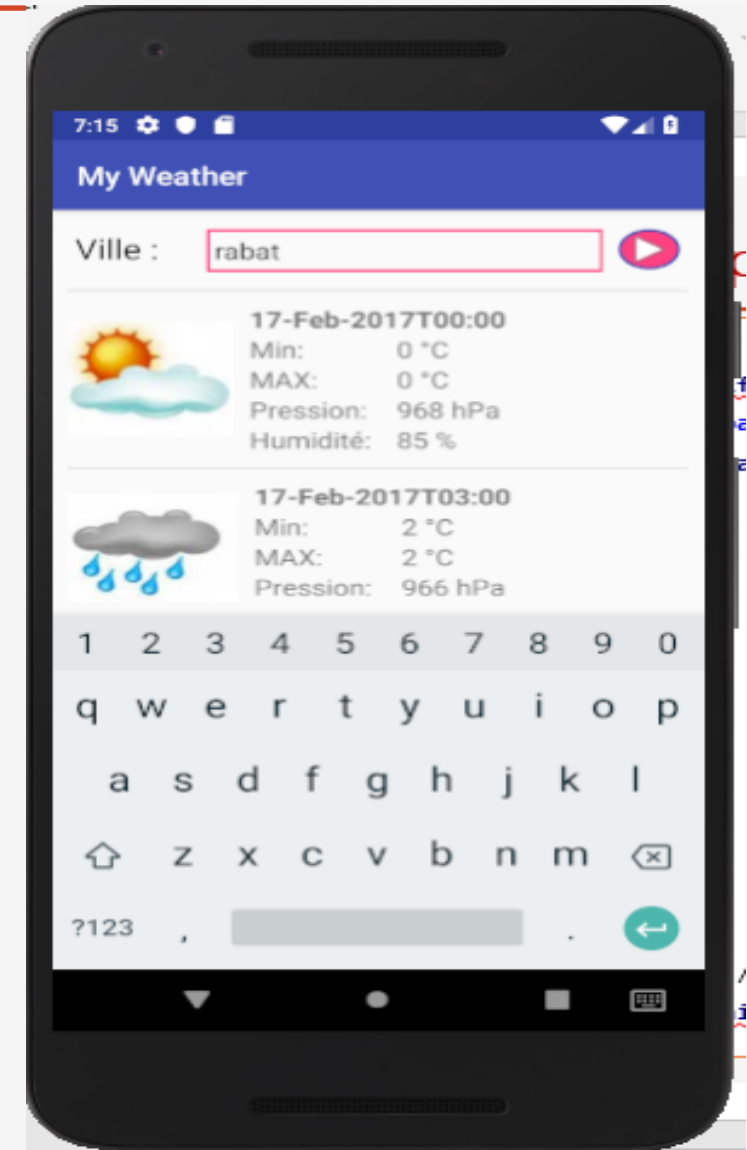


Activité n° 2 : AndroidManifest.xml

```
• <?xml version="1.0" encoding="utf-8"?>
  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="net.youssfi.httpandroidapp">
    <uses-permission android:name="android.permission.INTERNET" />

    <application
      android:allowBackup="true"
      android:icon="@mipmap/ic_launcher"
      android:label="@string/app_name"
      android:roundIcon="@mipmap/ic_launcher_round"
      android:supportRtl="true"
      android:theme="@style/AppTheme">
      <activity android:name=".MainActivity">
        <intent-filter>
          <action android:name="android.intent.action.MAIN" />
          <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
      </activity>
    </application>

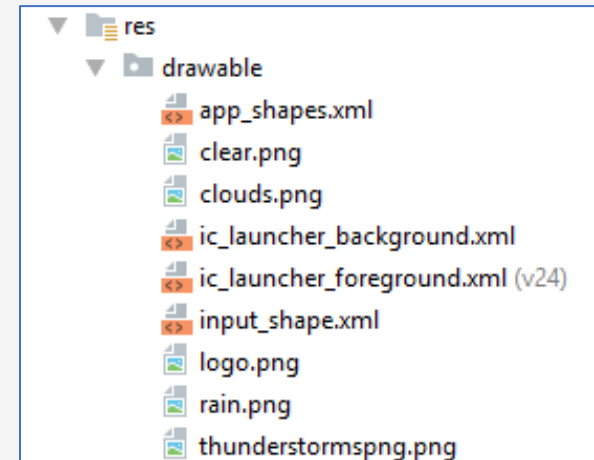
  </manifest>
```



Activité n° 2 : app_shape.xml et input_shape.xml

```
• <?xml version="1.0" encoding="utf-8"?>
  <shape xmlns:android="http://schemas.android.com/apk/res/android"
        android:shape="oval">
    <stroke android:width="2dp" android:color="@color/colorPrimary"
    ></stroke>
    <solid android:color="@color/colorAccent"></solid>
  </shape>
```

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
      android:shape="rectangle">
    <stroke android:width="2dp" android:color="@color/colorAccent" ></stroke>
</shape>
```

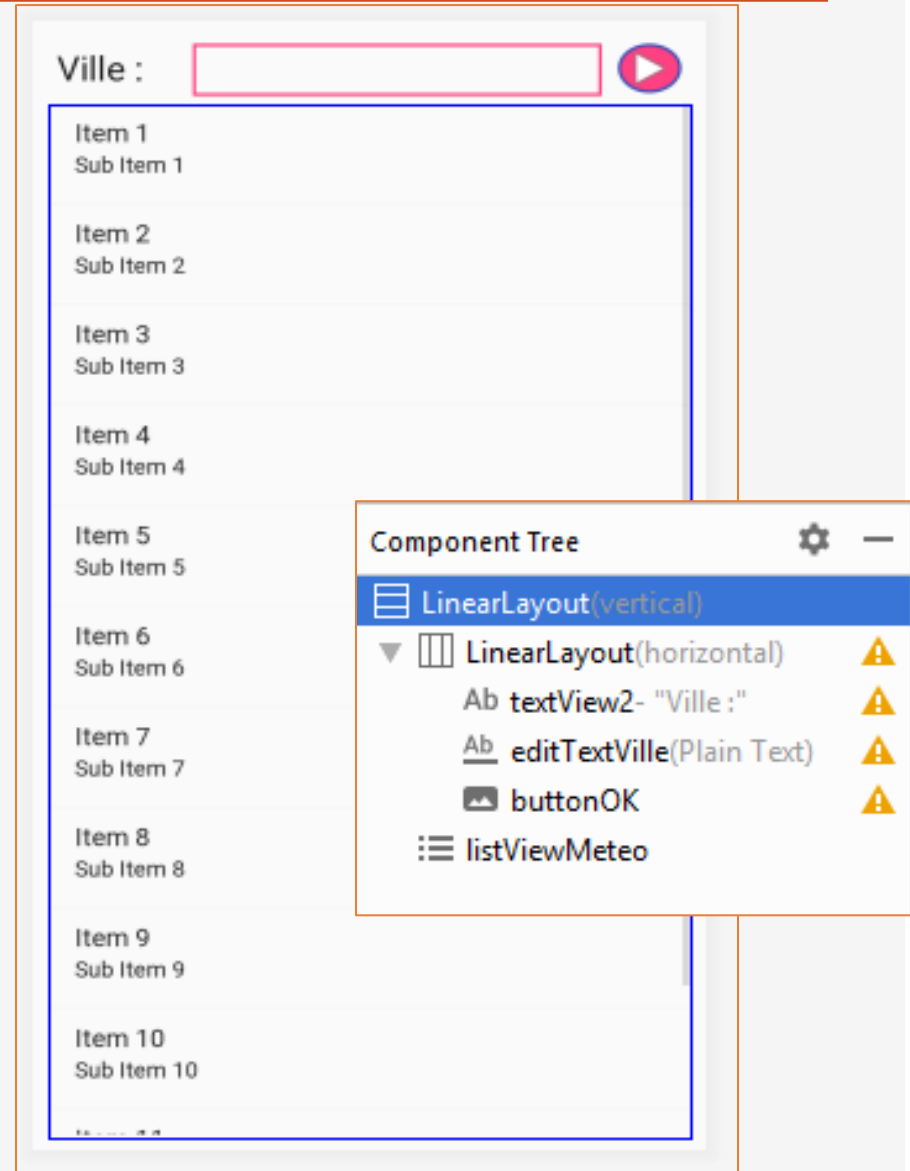


Ville :

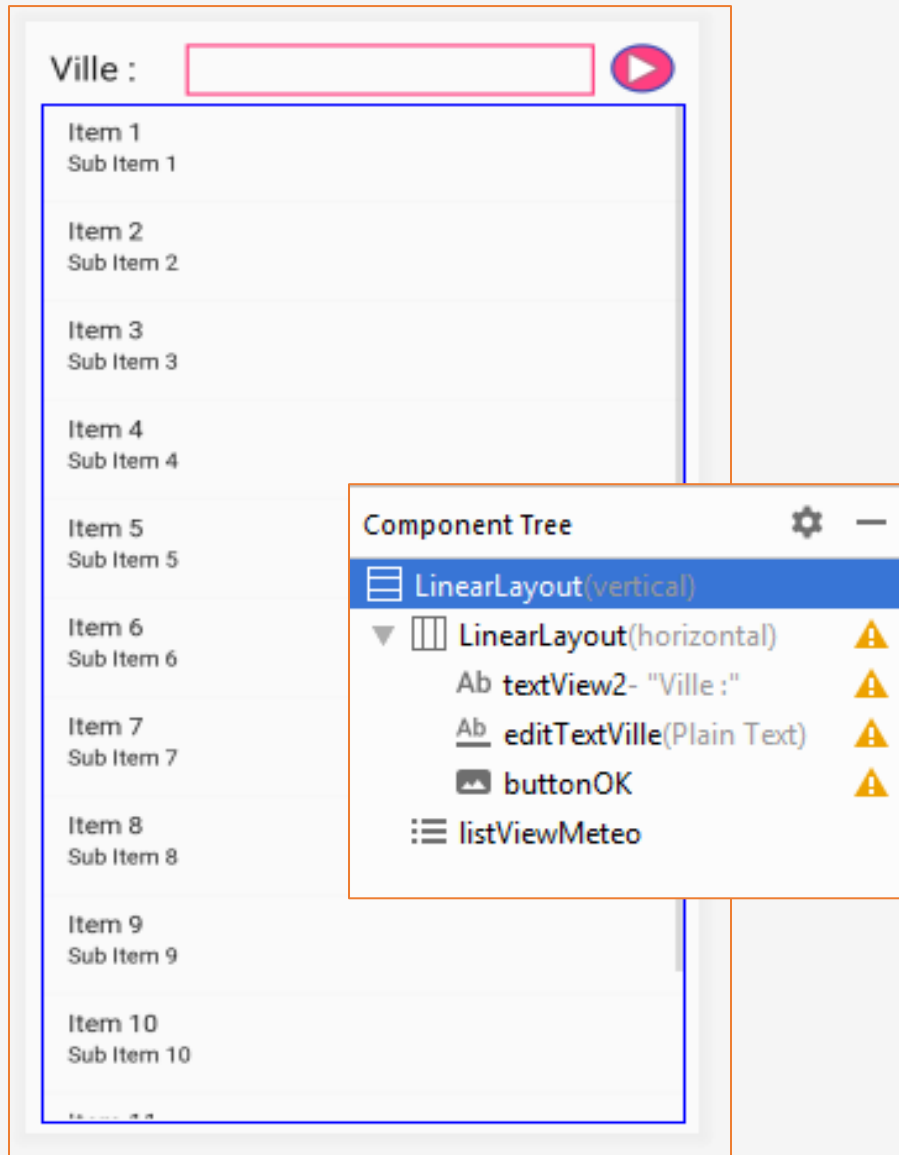


Activité n° 2 : activity_main.xml

```
• <?xml version="1.0" encoding="utf-8"?>
  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity" android:padding="10dp">
    <LinearLayout
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:orientation="horizontal">
      <TextView
        android:id="@+id/textView2"
        android:layout_width="73dp"
        android:layout_height="wrap_content"
        android:text="Ville :"
        android:textAppearance="@style/TextAppearance.AppCompat.Large"
        android:layout_margin="5dp"/>
    </LinearLayout>
```



Activité n° 2 : activity_main.xml



<EditText

```
android:id="@+id/editTextVille"  
android:layout_width="250dp"  
android:layout_height="wrap_content"  
android:layout_margin="5dp"  
android:background="@drawable/input_shape"  
android:ems="10"  
android:inputType="textPersonName"  
android:padding="5dp" />
```

<ImageButton

```
android:id="@+id/buttonOK"  
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:layout_weight="1"  
app:srcCompat="@android:drawable/ic_media_play"  
android:background="@drawable/app_shapes"  
android:layout_margin="5dp" />/>
```

</LinearLayout>

<ListView

```
android:id="@+id/listViewMeteo"  
style="@android:style/Widget.Material.ListView"  
android:layout_width="match_parent"  
android:layout_height="wrap_content" />
```


Activité n° 2 : list_item_layout.xml

list_item_layout.xml x MeteoListModel.java x MeteoItem.java x app x activity_main.xml x app_shapes.xml x

Component Tree

- LinearLayout(horizontal)
 - imageView
 - LinearLayout(vertical)
 - Ab textViewDate
 - LinearLayout(horizontal)
 - Ab textView- "Min:"
 - Ab textViewTempMin
 - LinearLayout(horizontal)
 - Ab textView4- "MAX:"
 - Ab textViewTempMAX
 - LinearLayout(horizontal)
 - Ab textView6- "Pression:"
 - Ab textViewPression
 - LinearLayout(horizontal)
 - Ab textView9- "Humidité:"
 - Ab textViewHumidite

Pixel P AppTheme 50%



Min:	
MAX:	
Pression:	
Humidité:	

Activité n° 2 : list_item_layout.xml

- ```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="wrap_content">
<ImageView
 android:id="@+id/imageView"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:layout_gravity="center_vertical"
 tools:srcCompat="@tools:sample/avatars" />
```

```
<LinearLayout
```

```
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:orientation="vertical"
 android:padding="10dp">
```

```
<TextView
```

```
 android:id="@+id/textViewDate"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:textAlignment="textStart"
 android:textAppearance="@style/TextAppearance.AppCompat.Medium"
 android:textStyle="bold" />
```

## Activité n° 2 : list\_item\_layout.xml

- <LinearLayout

```
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:orientation="horizontal">
 <TextView
 android:id="@+id/textView"
 android:layout_width="93dp"
 android:layout_height="wrap_content"
 android:text="Min:"
 android:textAlignment="textStart"
 android:textAppearance="@style/TextAppearance.AppCompat.Medium" />
```

- <TextView

```
 android:id="@+id/textViewTempMin"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:layout_weight="1"
 android:textAlignment="textStart"
 android:textAppearance="@style/TextAppearance.AppCompat.Medium" />
</LinearLayout>
```

## Activité n° 2 : list\_item\_layout.xml

```
• <LinearLayout
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:orientation="horizontal">
 <TextView
 android:id="@+id/textView4"
 android:layout_width="93dp"
 android:layout_height="match_parent"
 android:text="MAX: "
 android:textAlignment="textStart"
 android:textAppearance="@style/TextAppearance.AppCompat.Medium" />

 <TextView
 android:id="@+id/textViewTempMAX"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:layout_weight="1"
 android:textAlignment="textStart"
 android:textAppearance="@style/TextAppearance.AppCompat.Medium" />
 </LinearLayout>
```

## Activité n° 2 : list\_item\_layout.xml

- <LinearLayout

```
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:orientation="horizontal">
 <TextView
 android:id="@+id/textView6"
 android:layout_width="93dp"
 android:layout_height="wrap_content"
 android:text="Pression:"
 android:textAlignment="textStart"
 android:textAppearance="@style/TextAppearance.AppCompat.Medium" />
```

- <TextView

```
 android:id="@+id/textViewPression"
 android:layout_width="166dp"
 android:layout_height="wrap_content"
 android:textAlignment="textStart"
 android:textAppearance="@style/TextAppearance.AppCompat.Medium" />
</LinearLayout>
```

## Activité n° 2 : list\_item\_layout.xml

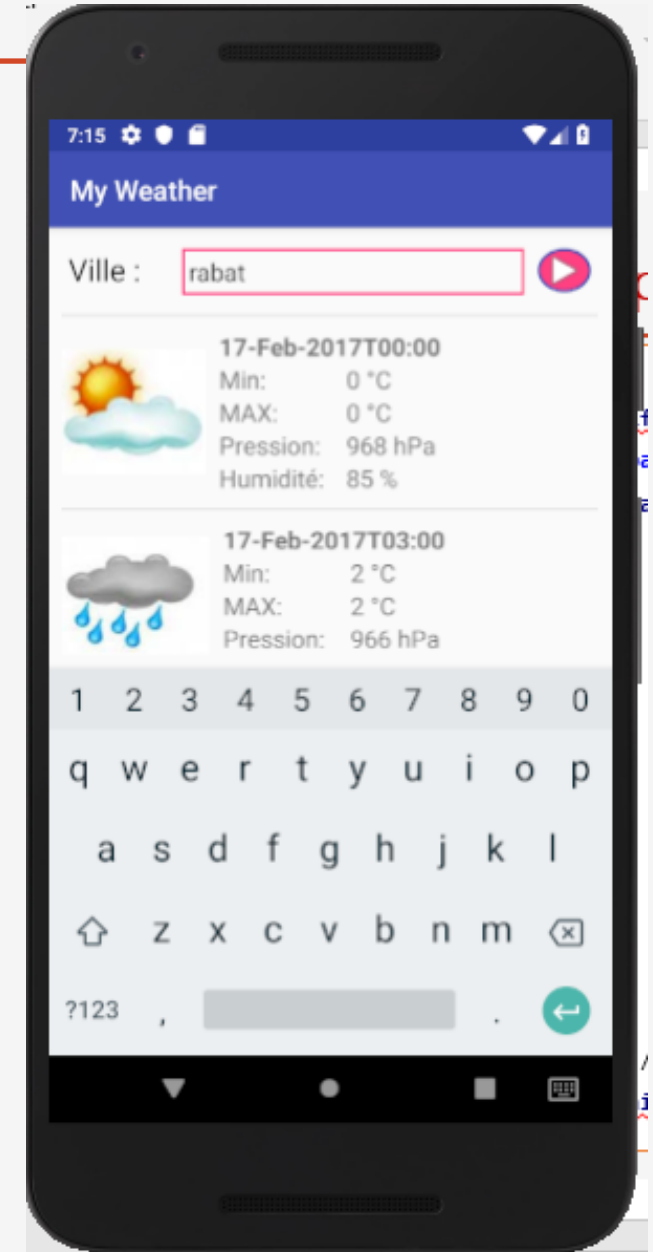
```
• <LinearLayout
 android:layout_width="match_parent"

 android:layout_height="wrap_content"
 android:orientation="horizontal">
 <TextView
 android:id="@+id/textView9"
 android:layout_width="93dp"
 android:layout_height="wrap_content"
 android:text="Humidité:"
 android:textAlignment="textStart"
 android:textAppearance="@style/TextAppearance.AppC
ppCompat.Medium" />
```

```
<TextView
 android:id="@+id/textViewHumidite"
 android:layout_width="match_parent"
 android:layout_height="wrap_content"
 android:textAlignment="textStart"
 android:textAppearance="@style/TextAppearance.AppC
ompat.Medium" />
</LinearLayout>
</LinearLayout>
</LinearLayout>
```

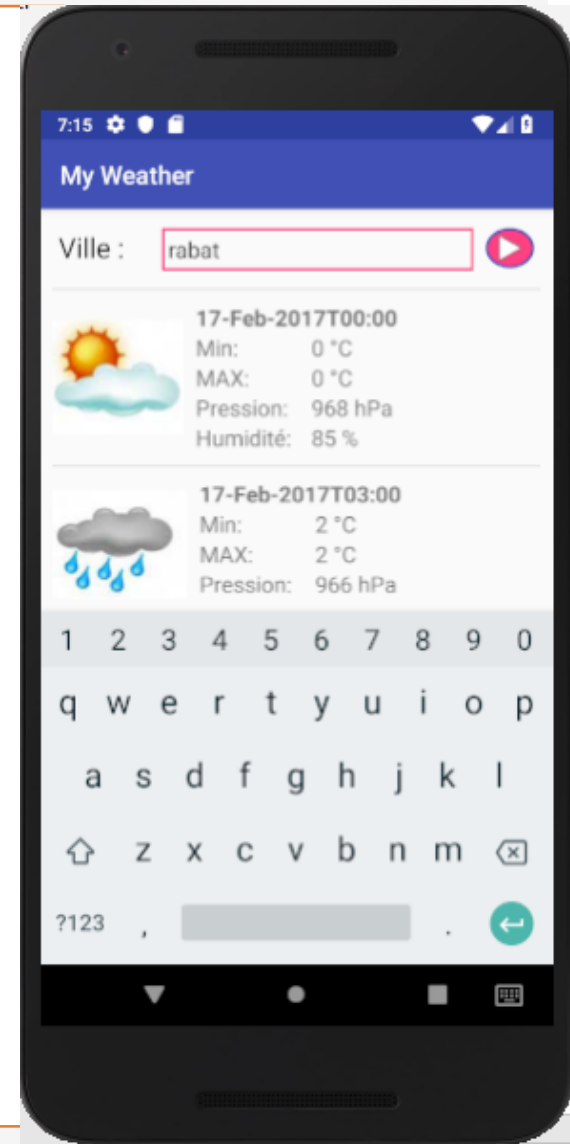
# MainActivity.java

```
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.EditText;
import android.widget.ImageButton;
import android.widget.ListView;
import com.android.volley.Request;
import com.android.volley.RequestQueue;
import com.android.volley.Response;
import com.android.volley.VolleyError;
import com.android.volley.toolbox.StringRequest;
import com.android.volley.toolbox.Volley;
import org.json.JSONArray;
import org.json.JSONException; import org.json.JSONObject;
import java.text.SimpleDateFormat; import java.util.ArrayList;
import java.util.Date;
import java.util.List;
```



# MainActivity.java

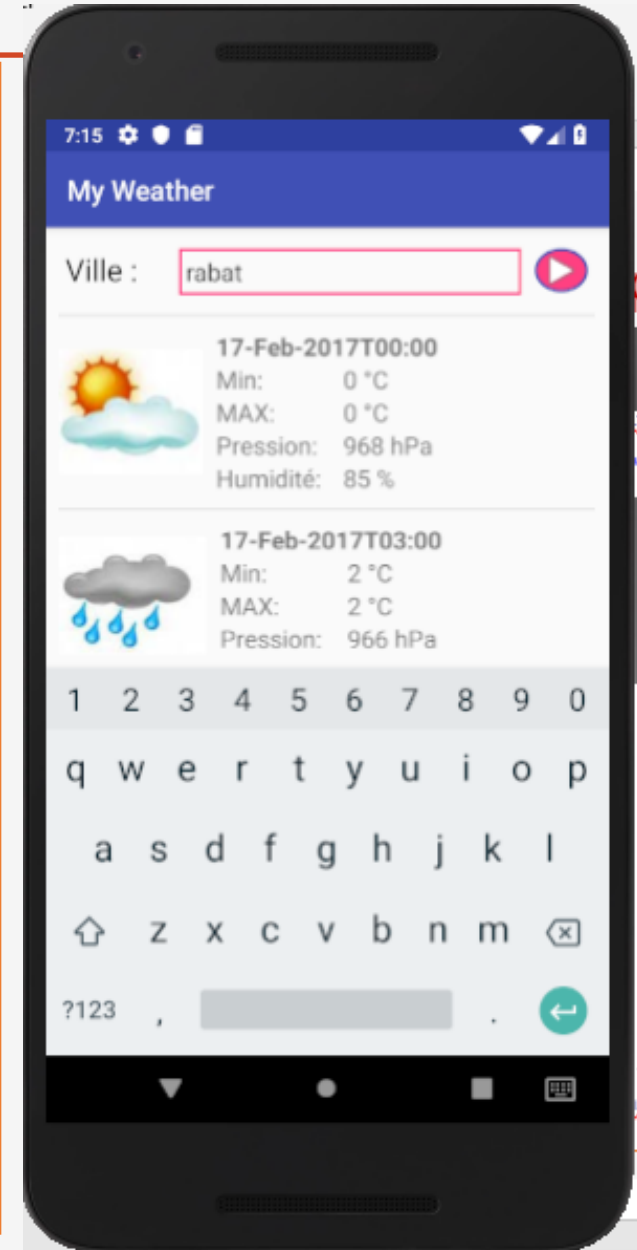
```
public class MainActivity extends AppCompatActivity {
 private EditText editTextVille; private ListView listViewMeteo;
 //private List<String> data=new ArrayList<>();
 List<MeteoItem> data=new ArrayList<>();
 //private ArrayAdapter<String> model;
 private MeteoListModel model; private ImageButton buttonOK;
 @Override
 protected void onCreate(Bundle savedInstanceState) {
 super.onCreate(savedInstanceState);
 setContentView(R.layout.activity_main);
 editTextVille=findViewById(R.id.editTextVille);
 listViewMeteo=findViewById(R.id.listViewMeteo);
 buttonOK=findViewById(R.id.buttonOK);
 model=new MeteoListModel(getApplicationContext(),R.layout.list_item_layout,data);
 listViewMeteo.setAdapter(model);
 //model=new ArrayAdapter<>(this,android.R.layout.simple_list_item_1,data);
 }
}
```





# MainActivity.java

```
buttonOK.setOnClickListener(new View.OnClickListener() {
 @Override
 public void onClick(View v) {
 Log.i("MyLog", ".....");
 data.clear(); model.notifyDataSetChanged();
 RequestQueue queue = Volley.newRequestQueue(getApplicationContext());
 String ville=editTextVille.getText().toString();
 Log.i("MyLog",ville);
 String url =
 "https://samples.openweathermap.org/data/2.5/for
 ecast?q="+ville+"&appid=a4578e39643716894ec78b28
 a71c7110";
 StringRequest stringRequest = new StringRequest(Request.Method.GET, url,
 new Response.Listener<String>() {
```



# MainActivity.java

```
@Override
public void onResponse(String response) {
 try {
 Log.i("MyLog", "-----");
 Log.i("MyLog", response);
 List<MeteoItem> meteoItems=new ArrayList<>();
 JSONObject jsonObject=new JSONObject(response);
 JSONArray jsonArray=jsonObject.getJSONArray("list");
 for (int i=0;i<javascriptArray.length();i++){
 MeteoItem meteoItem=new MeteoItem();
 JSONObject d=jsonArray.getJSONObject(i);
 Date date=new Date(d.getLong("dt")*1000);
 SimpleDateFormat sdf = new SimpleDateFormat("dd-MMM-yyyy'T'HH:mm");
 String dateString = sdf.format(date);
 JSONObject main=d.getJSONObject("main");
 JSONArray weather=d.getJSONArray("weather");
 int tempMin=(int)(main.getDouble("temp_min")-273.15);
 int tempMax=(int)(main.getDouble("temp_max")-273.15);
 }
 }
}
```

```
{
 "cod": "200",
 "message": 0.0032,
 "cnt": 36,
 "list": [
 {
 "dt": 1487246400,
 "main": {
 "temp": 286.67,
 "temp_min": 281.556,
 "temp_max": 286.67,
 "pressure": 972.73,
 "sea_level": 1046.46,
 "grnd_level": 972.73,
 "humidity": 75,
 "temp_kf": 5.11
 },
 "weather": [
 {
 "id": 800,
 "main": "Clear",
 "description": "clear sky",
 "icon": "01d"
 }
],
 "clouds": {
 "all": 0
 },
 "wind": {
 "speed": 1.81,
 "deg": 247.501
 },
 "sys": {
 "pod": "d"
 },
 "dt_txt": "2017-02-16 12:00:00"
 }
],
}
```

```
int pression=main.getInt("pressure");
int humidity=main.getInt("humidity");
//data.add(String.format(dateString+"\n"+"Min=%d °c, Max=%d °C",tempMin,tempMax));
meteoItem.tempMax=tempMax; meteoItem.tempMin=tempMin;
meteoItem.pression=pression; meteoItem.humidite=humidity;
meteoItem.date=dateString;
meteoItem.image=weather.getJSONObject(0).getString("main");
meteoItems.add(meteoItem);
data.add(meteoItem);
}
model.notifyDataSetChanged();
} catch (JSONException e) {
 e.printStackTrace();
}
}
},
},
```

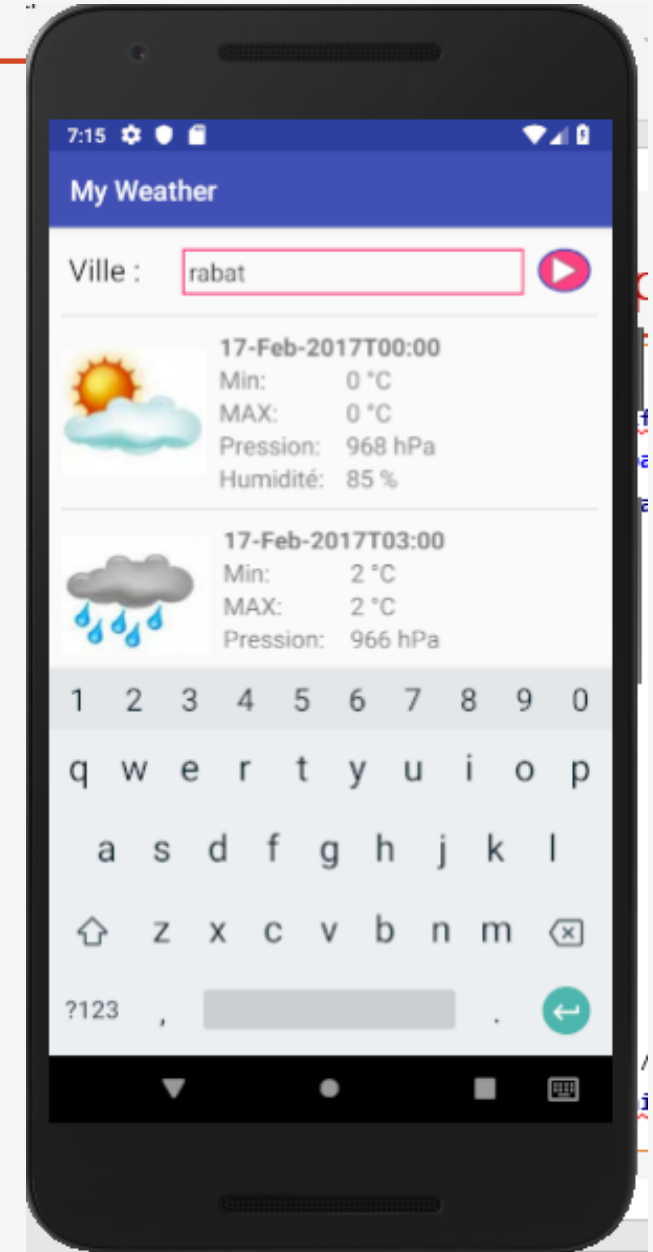
```
{
 "cod": "200",
 "message": 0.0032,
 "cnt": 36,
 "list": [
 {
 "dt": 1487246400,
 "main": {
 "temp": 286.67,
 "temp_min": 281.556,
 "temp_max": 286.67,
 "pressure": 972.73,
 "sea_level": 1046.46,
 "grnd_level": 972.73,
 "humidity": 75,
 "temp_kf": 5.11
 },
 "weather": [
 {
 "id": 800,
 "main": "Clear",
 "description": "clear sky",
 "icon": "01d"
 }
],
 "clouds": {
 "all": 0
 },
 "wind": {
 "speed": 1.81,
 "deg": 247.501
 },
 "sys": {
 "pod": "d"
 },
 "dt_txt": "2017-02-16 12:00:00"
 }
],
}
```

# MainActivity.java

```
new Response.ErrorListener() {
 @Override
 public void onErrorResponse(VolleyError error) {
 Log.i("MyLog", "Connection problem!");
 }
});
```

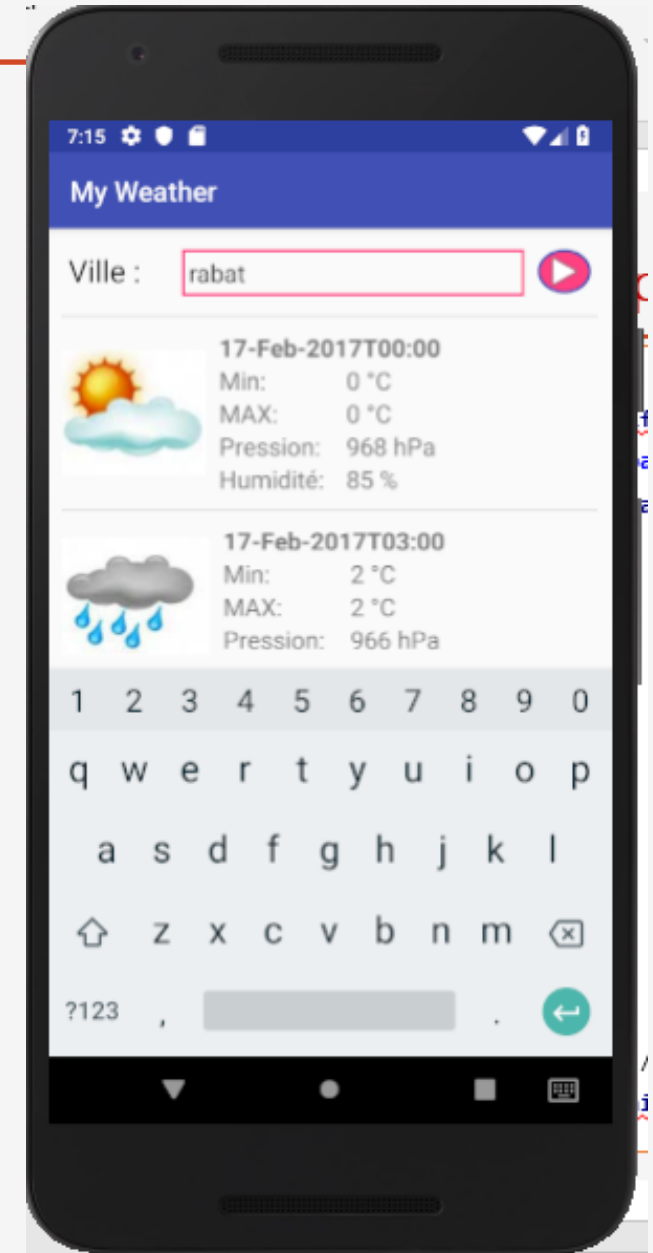
```
// Add the request to the RequestQueue.
```

```
queue.add(stringRequest);
}
 });
}
}
```



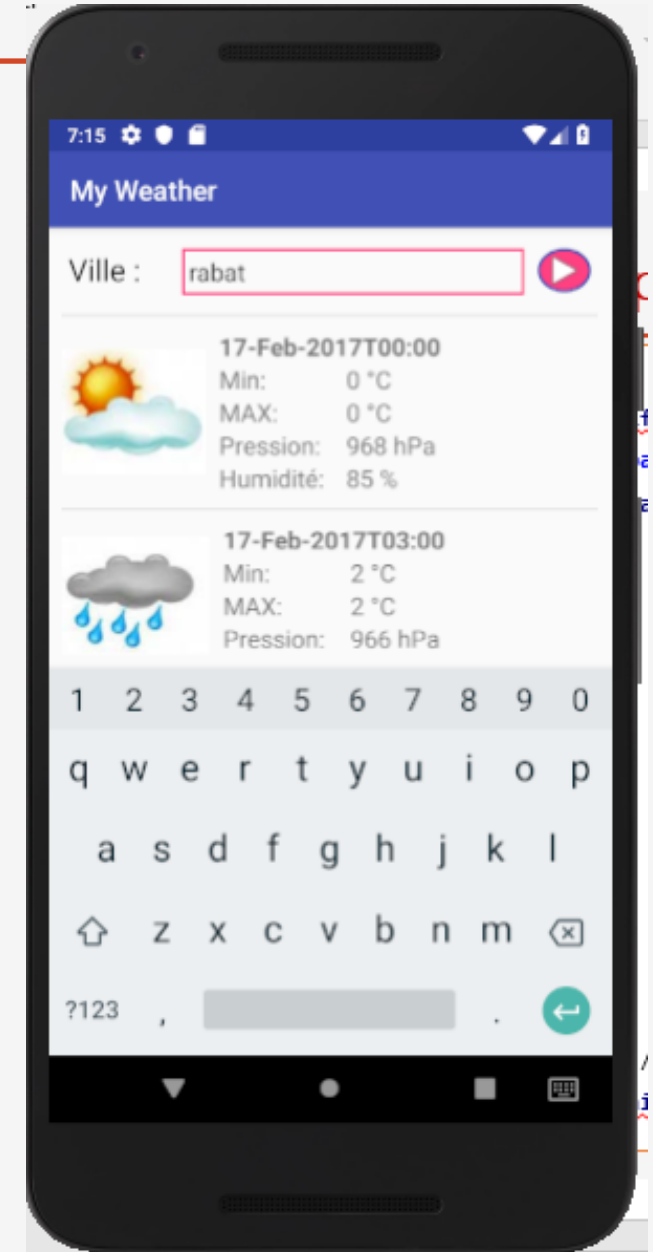
# MeteoItem.java

```
import java.util.Date;
public class MeteoItem {
 public int tempMax;
 public int tempMin;
 public int pression;
 public int humidite;
 public String image;
 public String date;
}
```



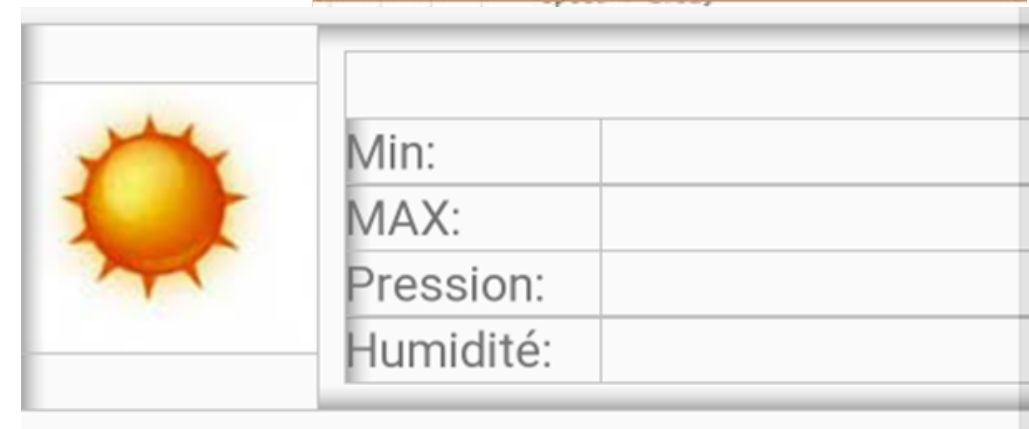
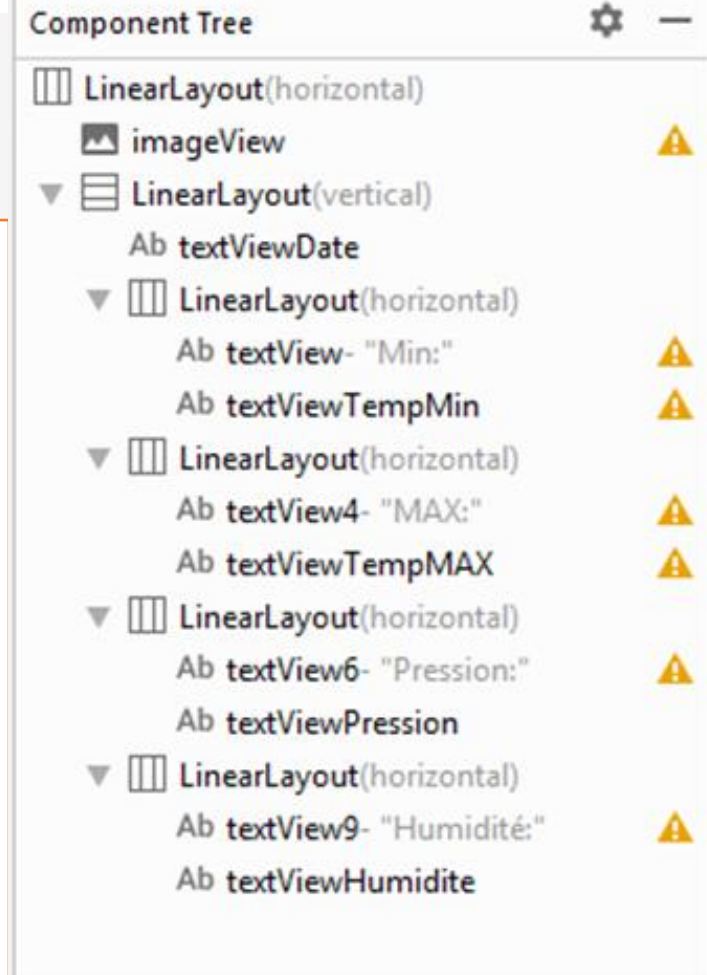
# MeteoListModel.java

```
import android.content.Context;
import android.support.annotation.NonNull;
import android.support.annotation.Nullable;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ArrayAdapter;
import android.widget.ImageView; import android.widget.TextView;
import java.util.HashMap; import java.util.List;
import java.util.Map;
```



# MeteoListModel.java

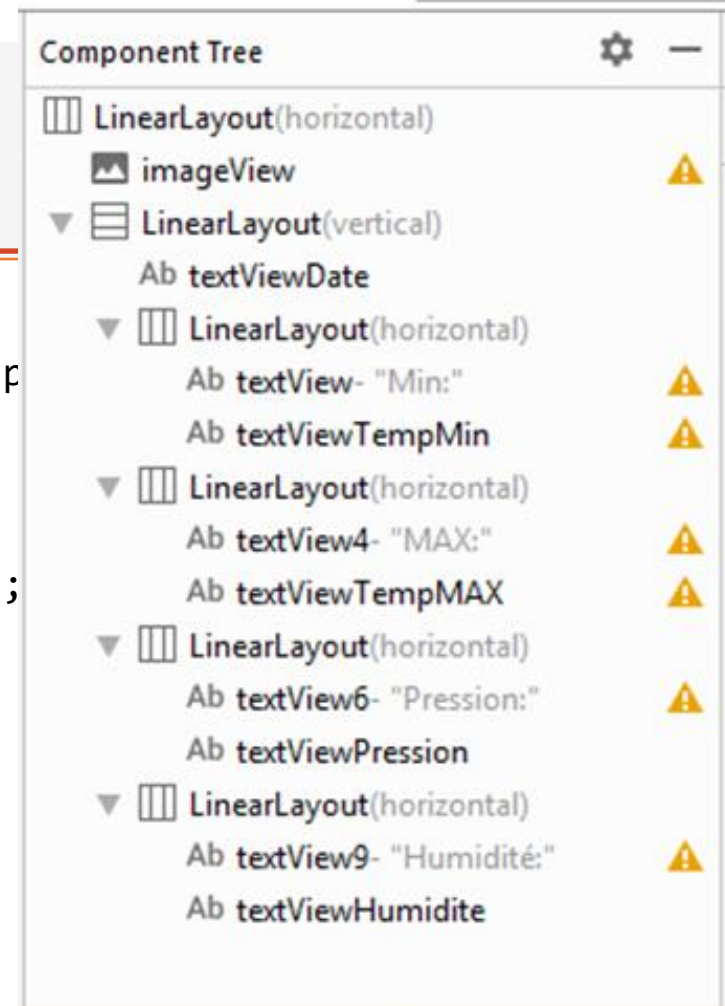
```
public class MeteoListModel extends ArrayAdapter<MeteoItem> {
 private List<MeteoItem> listItems; private int resource;
 public static Map<String,Integer> images=new HashMap<>();
 static{
 images.put("Clear",R.drawable.clear);
 images.put("Clouds",R.drawable.clouds);
 images.put("Rain",R.drawable.rain);
 images.put("thunderstormpng",R.drawable.thunderstormpng);
 }
 public MeteoListModel(@NonNull Context context,int resource,
List<MeteoItem> data) {
 super(context, resource,data);
 Log.i("Size:",String.valueOf(data.size()));
 this.listItems=data; this.resource=resource;
 }
}
```





# MeteoListModel.java

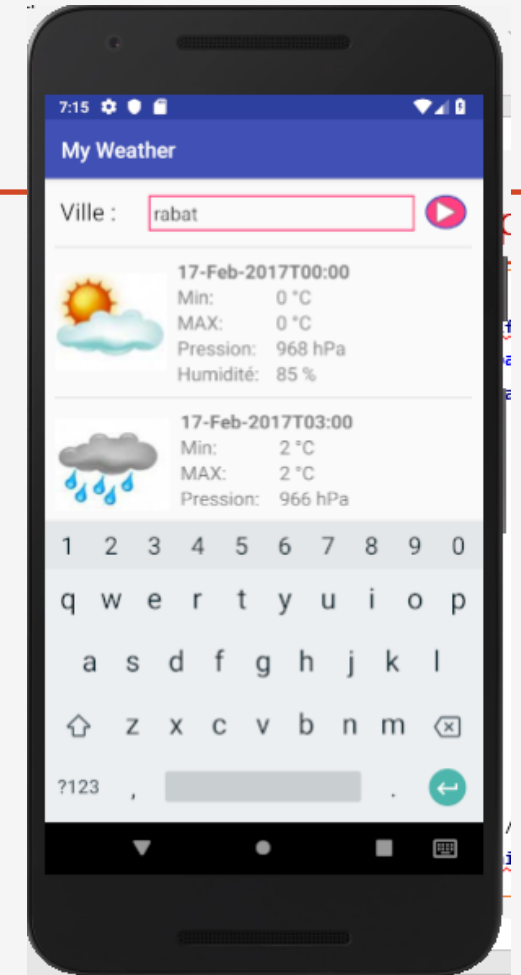
```
• @NonNull
• @Override
• public View getView(int position, @Nullable View convertView, @NonNull ViewGroup
parent) {
• Log.i("MyLog", "....."+position);
• View listItem=convertView;
• if(listItem==null)
• listItem =LayoutInflater.from(getContext()).inflate(resource,parent,false);
• ImageView imageView=listItem.findViewById(R.id.imageView);
• TextView textViewTempMax=listItem.findViewById(R.id.textViewTempMAX);
• TextView textViewTempMin=listItem.findViewById(R.id.textViewTempMin);
• TextView textViewPression=listItem.findViewById(R.id.textViewPression);
• TextView textViewHumidite=listItem.findViewById(R.id.textViewHumidite);
• TextView textViewDate=listItem.findViewById(R.id.textViewDate);
• String key=listItems.get(position).image;
```





# MeteoListModel.java

- `if(key!=null) imageView.setImageResource(images.get(key));`
- `textViewTempMax.setText(String.valueOf(listItems.get(position).tempMax)+" °C");`
- `textViewTempMin.setText(String.valueOf(listItems.get(position).tempMin)+" °C");`
- `textViewPression.setText(String.valueOf(listItems.get(position).pression)+" hPa");`
- `textViewHumidite.setText(String.valueOf(listItems.get(position).humidite)+" %");`
- `textViewDate.setText(String.valueOf(listItems.get(position).date));`
- `return listItem;`
- `}`
- `}`



Min:	
MAX:	
Pression:	
Humidité:	



Jetpack  
Compose

---

# JetPack Compose



# JetPack Compose

## Déclarative

```
class MainActivity : ComponentActivity() {
 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContent {
 TAC_cours_ComposeTheme {
 // A surface container using the 'background' color from the theme
 Surface(
 modifier = Modifier.fillMaxSize(),
 color = MaterialTheme.colorScheme.background
) {
 Greeting("Android")
 }
 }
 }
 }
}

@Composable
fun Greeting(name: String) {
 Text(text = "Hello $name!")
}
```

## Programmatic

```
class MainActivity : AppCompatActivity() {
 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContentView(R.layout.activity_main)
 findViewById(R.id.monTexte).setText(...)
 }
}
```

---

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout "
 xmlns:app="http://schemas.android.com/apk/res-auto"
 xmlns:tools="http://schemas.android.com/tools"
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 tools:context=".MainActivity">

 <TextView
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:text="Hello World!"
```

# JetPack Compose

## Déclarative

```
class MainActivity : ComponentActivity() {
 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContent {
 TAC_cours_ComposeTheme {
 // A surface container using the 'background' color from the theme
 Surface(
 modifier = Modifier.fillMaxSize(),
 color = MaterialTheme.colorScheme.background
) {
 Greeting("Android")
 }
 }
 }
 }
}
```

**@Composable**

```
fun Greeting(name: String) {
 Text(text = "Hello $name!")
}
```

2022-2023

## Programmatic

```
class MainActivity : AppCompatActivity() {
 override fun onCreate(savedInstanceState: Bundle?) {
 super.onCreate(savedInstanceState)
 setContentView(R.layout.activity_main)
 findViewById(R.id.monTexte).setText(...)
 }
}
```

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout "
 xmlns:app="http://schemas.android.com/apk/res-auto"
 xmlns:tools="http://schemas.android.com/tools"
 android:layout_width="match_parent"
 android:layout_height="match_parent"
 tools:context=".MainActivity">
```

```
<TextView
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:text="Hello World!"
```